

GALAS Signature Scheme

Submission to ICCS NGCC

Guoxiao Liu¹ Zhuoqing Peng¹ Chongxu Ren¹
Guowei Liu² Juntong Lin¹ Yimeng Sun²
Keting Jia^{1,*} Hongbo Yu¹

*Corresponding submitter: Keting Jia
Tsinghua University, Beijing, P.R. China
ktjia@tsinghua.edu.cn

¹Tsinghua University, Beijing, P.R. China

²Shandong University, Qingdao, P.R. China

June 26, 2026

Contents

1	Introduction	1
1.1	Security Claim	1
1.2	Design Target and Scope	2
1.3	Organization	2
2	Notation and Conventions	2
3	High-Level Description	4
3.1	Scheme Overview	4
3.2	Design Rationale of GALA	5
4	Preliminaries and Primitives	8
4.1	Mathematical Preliminaries and Encoding	8
4.2	Symmetric Primitives	10
5	The GALA OWF and Constraint Relation	12
5.1	The GALA One-Way Function	12
5.2	Subfield Compression	14
5.3	Degree-3 Constraint System	15
6	VOLEitH and QuickSilver Construction	20
6.1	Batch All-But-One Vector Commitment (BAVC)	20
6.2	VOLE Commitment and Reconstruction	22
6.3	VOLE Consistency Hash	24
6.4	Fiat–Shamir Challenges and Grinding	25
6.5	QuickSilver Interface	26
7	Signature Scheme and Serialization	26
7.1	Key Generation	26
7.2	Signing	26
7.3	Verification	28
8	Parameters, Tests, and Performance	28
8.1	Parameters	28
8.2	Performance Evaluation	29
8.3	KAT and Test Vectors	31
9	Security Analysis	32
9.1	Security Claim and Scope	32
9.2	Security goal and proof boundary	32
9.3	Assumptions	33
9.4	Inherited reduction shape	33
9.5	Role of $\text{Good}(k, x)$	34
9.6	Correctness and failure probability	34
9.7	Concrete accounting of protocol parameters	35
9.8	Soundness of the GALAS relation	35
9.9	GALA one-wayness analysis	36
9.10	Quantum security interpretation	39

10 Algorithm Characteristics	40
10.1 Innovation	40
10.2 Simplicity	40
10.3 Flexibility	40
10.4 Compatibility	40
10.5 Scalability	41
10.6 Implementation Performance	41
A Implementation Notes	41
A.1 Revision history	41
A.2 Reference implementation	41
A.3 Optimized implementation	41
A.4 Performance comparison	42
A.5 Code organization	42
A.6 Build instructions	42
A.7 Implementation scope	43
B Public Constant Tables	43
B.1 Canonical Serialization	43
B.2 Round Constants	43
B.3 Machine-Readable Files and Hashes	44

1 Introduction

Digital signatures are a central part of the post-quantum transition. Shor’s algorithm rules out relying on the classical hardness of factoring and discrete logarithms in the long term [Sho94]. The NIST post-quantum standardization process has therefore produced a lattice-based signature standard [Nat24a] and a separate call for additional signature schemes [Nat24b]. This has left room for signature designs based on symmetric, code-based, multivariate, and MPC-in-the-Head techniques, each of which gives a different balance between signature size, signing cost, verification cost, and assumption diversity.

The Next-generation Commercial Cryptographic Algorithms Program (NGCC) of the Institute of Commercial Cryptography Standards (ICCS) gives another reason to study signatures built from symmetric and algebraic primitives. Such schemes can offer small public keys, transparent bit-level implementations, and a security basis that is largely orthogonal to lattice assumptions. MPC-in-the-Head signatures start from a zero-knowledge proof of knowledge built from secure multiparty computation [IKOS07]. The Fiat–Shamir transform then makes the public-coin proof non-interactive [FS87]. ZKBoo, Picnic, and Banquet are representative examples of this line of work [GMO16, CDG+20, BNO+21].

GALAS is a VOLE-in-the-Head signature scheme based on the GALA keyed one-way function introduced in the GALAS design paper [LSP+26]. Its proof system follows the VOLEitH framework [BBdSG+23] and uses QuickSilver-style authenticated polynomial checks [YSWW21]. FAEST is the closest implementation-level template for the transcript structure and serialization style [BBB+25a]. The GALAS-specific contribution is the combination of

- a wide-then-narrow inversion-based OWF, called GALA;
- subfield-compressed inverse witnesses of length $\lambda/2$ for each wide inversion;
- a degree-3 QuickSilver constraint system for the resulting relation.

The VOLE commitment, BAVC opening, QuickSilver batching, and Fiat–Shamir machinery follow the VOLEitH/QuickSilver framework. This separation is important for reading the specification: GALA defines the algebraic statement, while the proof machinery defines how that statement is proved and transformed into a signature.

1.1 Security Claim

The submitted GALAS signature scheme is designed to provide existential unforgeability under adaptive chosen-message attack (EUF-CMA), in the random-oracle/quantum-random-oracle style used for VOLEitH signatures. The claim is conditional on the following layered assumptions:

1. finding an accepting witness for the exact single-pair degree-3 GALAS relation is infeasible for public keys generated by Algorithm 19;
2. the inherited VOLEitH, BAVC, and QuickSilver machinery is sound for degree-3 authenticated polynomial relations;
3. the NGCC `pseudoXOF` and `pseudohash` interfaces are modeled as domain-separated pseudorandom expansion and random-oracle interfaces with the output lengths specified in Section 4.2.2.

The per-key usage bound for this claim is at most 2^{80} distinct messages under one public key. Accordingly, the serialized pre-IV and the derived IV are fixed to $n_{iv} = 160$ bits in all submitted parameter sets. The claim is deliberately a signature claim for one public pair (x, y) , not a claim that GALA is a general-purpose block cipher or a multi-query PRP.

Table 1: Claimed NGCC security targets. The notation $a(b)$ means a bits against classical attacks and b bits under the usual Grover square-root baseline.

Instance family	λ	Claimed target	Variants
Galas-160	160	128(80)	S, F
Galas-256	256	256(128)	S, F
Galas-384	384	384(192)	S, F
Galas-512	512	512(256)	S, F

1.2 Design Target and Scope

This document specifies the NGCC submission of GALAS. The normative public key is

$$pk = x \parallel y \in \{0, 1\}^{2\lambda},$$

where $y = \text{GALA}_k(x)$, and the normative secret key is

$$sk = k \in \{0, 1\}^\lambda.$$

The signing relation is therefore

$$R_{\text{GALA}} = \{((x, y), k) : \text{Good}(k, x) = 1 \text{ and } y = \text{GALA}_k(x)\}.$$

The extended witness proved inside QuickSilver is not only k , but $w = (k, \sigma_0, \sigma_1, \sigma_2)$, where the σ_i are the compressed subfield witnesses for the three wide inversions.

The submitted parameter sets use $\lambda \in \{160, 256, 384, 512\}$ and provide two variants for each field size. The variant labels select different VOLE/BAVC opening schedules and therefore different signature-size/performance tradeoffs; the algebraic GALA relation is the same for both variants at a given λ .

1.3 Organization

Section 2 fixes notation and serialization conventions used throughout the document. Section 3 gives the abstract signature flow and the signature-size formula before the lower-level machinery is introduced. Section 4 specifies the field encoding and the NGCC hash/XOF interfaces. Section 5 defines the GALA OWF, subfield compression, and the degree-3 constraint system. Section 6 specifies the inherited VOLEitH, BAVC, and QuickSilver interfaces used by signing and verification. Section 7 gives the normative KeyGen, Sign, and Verify algorithms and their serialization. Section 8 records the submitted parameters, test-vector material, and performance methodology. Section 9 gives the security analysis, and Section 10 summarizes the algorithmic characteristics relevant to the NGCC submission.

2 Notation and Conventions

Table 2 summarizes the notation used throughout the specification. Local loop indices may be reused inside pseudocode, but the global symbols in this table are kept distinct.

The names GALA and GALAS are used with different meanings throughout the document. GALA denotes the keyed one-way function and its algebraic relation, while GALAS denotes the full signature scheme. Typewriter names such as Galas-256F denote concrete submitted instances. The normative hash names are the semantic names H_{msg} , H_{seed} , H_{iv} , H_{com} , H_{FS1} , H_{FS2} , and H_{FS3} from Table 5; source-code helper names are implementation details and are not normative.

Table 2: Notation summary.

Symbol	Type	Meaning
λ	$\mathbb{N}$	Security parameter and field extension degree
m	$\mathbb{N}$	Subfield degree = $\lambda/2$
d_{QS}	$\mathbb{N}$	QuickSilver degree bound = 3
$\mathbb{F}_{2^n}$	field	Finite field with 2^n elements
α	$\mathbb{F}_{2^\lambda}$	Polynomial-basis element, represented as the root of φ_λ
β	$\mathbb{F}_{2^{64}}$	Polynomial-basis element for the auxiliary 64-bit field used by the universal hashes
$\varphi_n(X)$	polynomial	Irreducible polynomial defining $\mathbb{F}_{2^n}$
Low_{64}	map	Take the low 64 polynomial-basis coordinates of an element of $\mathbb{F}_{2^\lambda}$ and interpret them in $\mathbb{F}_{2^{64}}$
ι_{64}	map	Zero-extend an element of $\mathbb{F}_{2^{64}}$ into the low 64 coordinates of $\mathbb{F}_{2^\lambda}$
$N(a)$	$\mathbb{F}_{2^m}$	Norm a^{2^m+1} from $\mathbb{F}_{2^\lambda}$ to $\mathbb{F}_{2^m}$
Frob_m	map	Frobenius map $a \mapsto a^{2^m}$
P	list	Bit permutation selecting pivot columns for the subfield embedding
$\mathcal{B}'$	set	Embedding basis for subfield compression
σ	$\{0, 1\}^m$	Compressed subfield witness
$\bar{\sigma}$	$\mathbb{F}_{2^m} \subset \mathbb{F}_{2^\lambda}$	Embedded subfield witness $\text{Embed}_{\mathcal{B}'}(\sigma)$, derived from σ rather than serialized separately
c_i	$\mathbb{F}_{2^\lambda}$	GALA round constants
M_i	map	Input linearized polynomials
L_i	map	Layer linearized polynomials
k	$\mathbb{F}_{2^\lambda}$	Secret key
x, y	$\mathbb{F}_{2^\lambda}$	Public key components
$\mathbf{w}$	$\{0, 1\}^\ell$	Extended witness
ℓ	$\mathbb{N}$	Witness length = $5\lambda/2$
$\hat{\ell}$	$\mathbb{N}$	Padded VOLE length = $\ell + d_{\text{QS}}\lambda + B$
B	$\mathbb{N}$	VOLE consistency slack, fixed to 16 bits
n_{iv}	$\mathbb{N}$	Pre-IV and IV length, fixed to 160 bits
τ	$\mathbb{N}$	Number of logical VOLE instances and BAVC subtrees
k_{tree}	$\mathbb{N}$	BAVC index depth for the first τ_1 logical subtrees
k_i	$\mathbb{N}$	BAVC index depth of logical subtree i
τ_1	$\mathbb{N}$	Number of depth- k_{tree} trees in the one-tree opening schedule
τ_0	$\mathbb{N}$	Number of depth- $(k_{\text{tree}} - 1)$ trees in the one-tree opening schedule
N_i	$\mathbb{N}$	Number of leaves in logical subtree i , equal to 2^{k_i}
L	$\mathbb{N}$	Total number of logical BAVC leaves, equal to $\sum_i N_i$
T_{open}	$\mathbb{N}$	Maximum number of revealed seeds
u	$\mathbb{F}_2^{\hat{\ell}}$	VOLE first output, used as the witness mask
V	$\{0, 1\}^{\hat{\ell} \times \lambda}$	Sender VOLE tag matrix; row r is a λ -bit field tag for QuickSilver and column j is the VOLEHash input $V[*, j]$
Q	$\{0, 1\}^{\hat{\ell} \times \lambda}$	Verifier VOLE tag matrix with the same row/column views as V
π_{VOLE}	$\{0, 1\}^{\lambda+B}$	Serialized VOLEHash proof value derived from the sender's binary VOLE secret u

Continued on next page

Symbol	Type	Meaning
T_{VOLE}	$(\{0, 1\}^{\lambda+B})^\lambda$	Non-serialized VOLEHash transcript for the tag columns V_j or Q_j
χ_{QS}	$\{0, 1\}^\lambda$	Non-serialized QuickSilver check value bound by H_{FS3}
θ_1, θ_2	$\{0, 1\}^\lambda$	Serialized QuickSilver response elements; $\pi_{\text{QS}} = \theta_1 \parallel \theta_2$
Δ	$\mathbb{F}_{2^\lambda}$	Global QuickSilver challenge point
$\hat{x}(\Delta)$	$\mathbb{F}_{2^\lambda}$	Authenticated value of x at Δ
Promote_3	notation	View a degree- ≤ 3 QuickSilver expression in the common degree-3 coefficient format
chall_i	bytes	Fiat-Shamir challenge i
w_{grind}	$\mathbb{N}$	Number of forced zero bits in the final challenge
c_{Good}	$\mathbb{R}$	Constant loss induced by the valid-key subset selected by Good
c	bytes	Serialized VOLE correction strings
π_{QS}	bytes	QuickSilver proof material
π_{BAVC}	bytes	Serialized BAVC opening
K_{root}	$\{0, 1\}^\lambda$	BAVC root seed derived by H_{seed}
h	$\{0, 1\}^{2\lambda}$	BAVC commitment root
iv^{pre}	$\{0, 1\}^{n_{iv}}$	Serialized pre-IV
iv	$\{0, 1\}^{n_{iv}}$	$IV = H_{iv}(iv^{\text{pre}})$
d	$\{0, 1\}^\ell$	Serialized masked witness $w \oplus u[0..\ell]$
ctr	uint32	Grinding counter
PRG	oracle	PRG instantiated by pseudoX0F
$H_{\text{msg}}, H_{\text{seed}}, \dots$	oracle	Domain-separated hash oracles instantiated by pseudohash ; see Table 5

3 High-Level Description

3.1 Scheme Overview

3.1.1 The GALAS Relation

For a fixed security parameter λ , let $F = \mathbb{F}_{2^\lambda}$ and let $m = \lambda/2$. A public key is the pair $pk = x \parallel y$ with $x, y \in F$. A secret key is $sk = k \in F$. The key-generation distribution samples (k, x) until the validity predicate **Good** (k, x) holds, and then sets

$$y = \text{GALA}_k(x).$$

The signature proves knowledge of k for this public pair. Internally, the VOLEitH proof does not commit only to k . It commits to the extended witness

$$w = (k, \sigma_0, \sigma_1, \sigma_2) \in \{0, 1\}^\ell, \quad \ell = \lambda + 3m = \frac{5}{2}\lambda,$$

where each $\sigma_i \in \{0, 1\}^m$ is the subfield-compressed inverse witness for one wide-round inversion of **GALA**.

3.1.2 Transcript Flow and Signature Contents

The signing algorithm follows the high-level transcript structure of VOLEitH signatures.

1. Hash the public key and message to obtain $\mu = H_{\text{msg}}(pk, M)$.
2. Derive K_{root} and a pre-IV value from k and μ , expand the BAVC/VOLE trees, and obtain the prover VOLE values (u, V) together with correction strings c .

3. Derive the first Fiat–Shamir challenge from μ , the BAVC root, the correction strings, and the IV.
4. Build the extended GALA witness w from pk and sk , mask it as $d = w \oplus u[0..\ell)$, and run the VOLE consistency hash `VOLEHash`.
5. Derive the second Fiat–Shamir challenge and run the degree-3 QuickSilver proof for the GALA constraints.
6. Grind the final challenge Δ until it satisfies the opening predicate and the BAVC opening uses at most T_{open} seeds.
7. Serialize the correction strings, the VOLE consistency proof, the masked witness, the QuickSilver proof, the BAVC opening, Δ , the pre-IV value, and the grinding counter.

Verification reverses this process. It parses the signature, recomputes μ and the IV, reconstructs the verifier VOLE view from the BAVC opening, checks the VOLE consistency hash, recomputes the QuickSilver check at Δ , and finally checks that the hash-derived Δ equals the serialized value and satisfies the grinding/opening rules.

3.1.3 Communication and Signature Size

The main reason for using GALA rather than a generic SPN relation is that the VOLEitH communication depends directly on the witness length and the QuickSilver degree. For the submitted GALAS relation,

$$\ell = \frac{5}{2}\lambda, \quad d_{QS} = 3, \quad B = 16 \text{ bits},$$

and the padded VOLE payload length is

$$\hat{\ell} = \ell + d_{QS}\lambda + B = \ell + 3\lambda + 16.$$

Here B is the 16-bit universal-hash tail used by the VOLE consistency check. It is measured in bits in the formula above.

The serialized signature is

$$\sigma = c \parallel \pi_{\text{VOLE}} \parallel d \parallel \pi_{\text{QS}} \parallel \pi_{\text{BAVC}} \parallel \Delta \parallel iv_{\text{pre}} \parallel ctr,$$

with component sizes shown in Table 3. Therefore

$$\begin{aligned} |\sigma| &= (\tau - 1)\hat{\ell} + (\lambda + B) + \ell + (d_{QS} - 1)\lambda \\ &\quad + (2\tau + T_{open})\lambda + \lambda + n_{iv} + 32 \text{ bits}. \end{aligned}$$

The short and fast variants differ only in the VOLE/BAVC parameters $(\tau, w_{grind}, T_{open})$ and hence in the terms involving τ and T_{open} . The algebraic witness length ℓ and QuickSilver degree $d_{QS} = 3$ are unchanged for the two variants at the same field size.

3.2 Design Rationale of GALA

GALA is a VOLEitH-friendly one-way function, not a general-purpose block cipher. The relevant cost metric is the cost of proving the relation inside QuickSilver and VOLEitH, not the number of classical encryption rounds. This is the same design pressure that led to proof-oriented symmetric primitives such as LowMC for MPC/FHE settings [ARS⁺15]. Later designs such as Rain and AIM targeted shorter MPCitH-style signatures [DKR⁺22, KHS⁺23], and Lynx follows a related VOLEitH-oriented route [JYC⁺26]. The difference is that GALA is built specifically around subfield compression and a global QuickSilver degree bound of $d_{QS} = 3$.

Table 3: Signature components and their bit lengths.

Component	Bit length	Purpose
c	$(\tau - 1)\hat{\ell}$	VOLE correction strings
π_{VOLE}	$\lambda + B$	VOLE consistency hash output
d	ℓ	masked extended witness $w \oplus u[0..\ell)$
π_{QS}	$(d_{\text{QS}} - 1)\lambda = 2\lambda$	serialized QuickSilver responses
π_{BAVC}	$(2\tau + T_{\text{open}})\lambda$	τ hidden leaf commitments, opened seeds, zero padding
Δ	λ	final Fiat-Shamir challenge
iv_{pre}	$n_{\text{iv}} = 160$	preimage for the VOLE/BAVC IV
ctr	32	grinding counter, little-endian

3.2.1 The VOLEitH cost model

In a FAEST-style VOLEitH signature, the signer commits to an extended witness and proves a low-degree algebraic relation at a Fiat-Shamir challenge point. Let ℓ be the committed witness length and let d_{QS} be the global QuickSilver authentication degree. The effective VOLE payload is

$$\hat{\ell} = \ell + (d_{\text{QS}} - 1)\lambda + (\lambda + B) = \ell + d_{\text{QS}}\lambda + B.$$

The term $(d_{\text{QS}} - 1)\lambda$ hides the non-leading coefficients of the batched degree- d_{QS} QuickSilver polynomial [YSWW21]. The term $(\lambda + B)$ is the VOLE consistency-hash overhead in the VOLEitH proof architecture [BBdSG+23]; the same accounting is used by the FAEST specification [BBB+25a]. Thus an efficient relation must optimize two quantities at the same time:

- the committed witness length ℓ ;
- the maximum authentication degree d_{QS} of the checked constraints.

Degree 2 is attractive for generic QuickSilver proofs, but it is too tight for the norm witness used by GALA inversion. Higher degrees give more freedom but add a full λ bits of VOLE payload per degree increment. GALA therefore fixes the central design target at $d_{\text{QS}} = 3$: just enough degree to certify compressed inversion witnesses, but low enough that the extra QuickSilver padding does not dominate the signature.

3.2.2 Compact witnesses from inversion

Let $F = \mathbb{F}_{2^\lambda}$ and $K = \mathbb{F}_{2^m}$ with $m = \lambda/2$. The finite-field norm

$$N_{F/K}(a) = a^{2^m+1} = a \cdot a^{2^m}$$

maps F^* to K^* [LN97]. For a non-zero inversion input a , define the embedded subfield witness

$$\bar{\sigma} = N_{F/K}(a)^{-1} \in K.$$

Then

$$a^{-1} = \bar{\sigma} \cdot a^{2^m}.$$

The signer commits to the $m = \lambda/2$ selected coordinates $\sigma = \text{Select}_P(\bar{\sigma})$ rather than to the full λ bits of a^{-1} . Correctness is checked after embedding the coordinates back to K :

$$\bar{\sigma} \cdot a^{2^m} \cdot a = 1.$$

This is the key algebraic fact behind GALAS. In the QuickSilver degree model, Frobenius maps such as $a \mapsto a^{2^m}$ are $\mathbb{F}_2$ -linear maps on the chosen field representation. They do not consume multiplication degree. The displayed identity is therefore a product of three degree-1 authenticated quantities, not a degree-4 relation disguised by batching.

3.2.3 Avoiding chained SPN degree growth

A standard substitution-permutation network repeatedly feeds one nonlinear layer into the next. That is efficient for many conventional symmetric-key designs, but it is a poor match for aggressive subfield compression. Suppose a first compressed inversion starts from a degree-1 authenticated value a . The reconstructed inverse $b = \bar{\sigma}a^{2^m}$ has authentication degree 2. If b is fed into another compressed inversion, the new consistency check would involve $\bar{\sigma}_b \cdot b^{2^m} \cdot b$ and would exceed the global $d_{QS} = 3$ budget.

The alternative would be to refresh the degree by committing to the intermediate full-field state after each nonlinear layer. That restores a low-degree input for the next S-box, but it loses the witness-size saving that subfield compression was meant to provide. GALA therefore avoids a long chain of compressed S-boxes.

3.2.4 Wide-then-narrow structure

GALA evaluates three inversions in parallel on values that are all $\mathbb{F}_2$ -linear functions of the key and public input:

$$a_0 = M_0(x \oplus k \oplus c_0), \quad a_1 = M_1(k \oplus c_1), \quad a_2 = M_2(k \oplus c_2).$$

Since each a_i has authentication degree 1, all three inversions admit the same half-size norm witness and the same degree-3 consistency check. Their outputs are mixed by public linear maps and narrowed to one field element

$$z = L_0(a_0^{-1}) + L_1(a_1^{-1}) + L_2(a_2^{-1}).$$

The public output is then tied to the final inverse by

$$y = z^{-1} \oplus L_3(k), \quad \text{equivalently} \quad z \cdot (y \oplus L_3(k)) = 1.$$

No extra witness is needed for this last inverse: y is public and $L_3(k)$ is linear in the committed key. Hence the extended witness remains

$$w = (k, \sigma_0, \sigma_1, \sigma_2), \quad \ell = \lambda + 3(\lambda/2) = 5\lambda/2.$$

3.2.5 Role of the linear maps

The public maps $M_0, M_1, M_2, L_0, L_1, L_2, L_3$ are invertible $\mathbb{F}_2$ -linear maps represented as linearized polynomials. Their first role is efficiency: applying such a map to authenticated values is a linear operation inside QuickSilver and does not increase the degree. Their second role is cryptanalytic separation. The three wide-round inversions are placed in independent public coordinate systems before being mixed, which is meant to prevent the low-dimensional decompositions that apply to more structured arithmetic-oriented primitives.

The semilinear family $x \mapsto \gamma x^{2^r}$ is the main degenerate case to keep in view. If a relative map $M_i M_j^{-1}$ were semilinear or very close to semilinear, an attacker could try to collapse two inverse branches into a simpler algebraic relation. The submitted maps are generated as dense invertible linear maps and are intended to behave like random elements of $\text{GL}_\lambda(2)$. Under this random-dense-map heuristic, exact semilinear coincidences and useful low-rank neighborhoods have negligible probability; nevertheless, the security discussion treats them as weak-instance conditions that should be audited for the fixed public constants [LSP⁺26].

4 Preliminaries and Primitives

4.1 Mathematical Preliminaries and Encoding

4.1.1 Binary extension fields

For each NGCC instance, the OWF field and the QuickSilver authentication field are the same field

$$F = \mathbb{F}_{2^\lambda}.$$

Elements of F are represented in the polynomial basis $1, \alpha, \alpha^2, \dots, \alpha^{\lambda-1}$, where α is a root of the irreducible polynomial $\varphi_\lambda(X)$ in Table 4. Addition in F is bitwise XOR. The notation follows standard finite-field conventions [LN97].

λ	defining polynomial $\varphi_\lambda(X)$
160	$X^{160} + X^5 + X^3 + X^2 + 1$
256	$X^{256} + X^{10} + X^5 + X^2 + 1$
384	$X^{384} + X^{16} + X^{15} + X^6 + 1$
512	$X^{512} + X^8 + X^5 + X^2 + 1$

Table 4: Field polynomials used by the NGCC GALAS instances.

4.1.2 Auxiliary 64-bit field

The universal hashes used by VOLEHash and ZKHash also use an auxiliary 64-bit field. This field is not one of the GALA OWF fields. It is defined as

$$\mathbb{F}_{2^{64}} = \mathbb{F}_2[\beta]/\varphi_{64}(\beta), \quad \varphi_{64}(X) = X^{64} + X^4 + X^3 + X + 1.$$

Elements of $\mathbb{F}_{2^{64}}$ are represented in the polynomial basis $1, \beta, \dots, \beta^{63}$. The serialization is the same little-endian coefficient serialization used for F : if $b = \sum_{j=0}^{63} b_j \beta^j$, then bit j of the eight-byte string is b_j , and bit 0 is the coefficient of β^0 . Multiplication in $\mathbb{F}_{2^{64}}$ is carry-less polynomial multiplication followed by reduction modulo $\varphi_{64}(X)$.

For $z = \sum_{j=0}^{\lambda-1} z_j \alpha^j \in F$, define

$$\text{Low}_{64}(z) = \sum_{j=0}^{63} z_j \beta^j \in \mathbb{F}_{2^{64}}.$$

Equivalently, $\text{Low}_{64}(z)$ is the field element represented by bytes $0, \dots, 7$ of the little-endian serialization of z . Conversely, define the coordinate zero-extension map

$$\iota_{64} \left(\sum_{j=0}^{63} b_j \beta^j \right) = \sum_{j=0}^{63} b_j \alpha^j \in F.$$

The map ι_{64} is an $\mathbb{F}_2$ -linear coordinate injection into the low 64 polynomial-basis coordinates of F ; it is not a field embedding and is not used for multiplication.

4.1.3 Byte and bit order

Field elements are serialized as little-endian byte strings. If $a = \sum_{j=0}^{\lambda-1} a_j \alpha^j$, then bit j of the serialized string is the coefficient a_j . Byte i contains bits $8i, \dots, 8i+7$, with bit $8i$ in the least significant bit position of that byte.

For bit strings, concatenation is written $\|$. The notation $s[i]$ denotes bit i with the same little-endian convention. When a bit string is interpreted as a field element, the polynomial-basis mapping above is used.

4.1.4 Subfield

For all NGCC parameters, λ is even. Let $m = \lambda/2$ and let

$$K = \mathbb{F}_{2^m} \subset F.$$

The subfield is the fixed field of the m -th Frobenius automorphism:

$$K = \{a \in F : a^{2^m} = a\}.$$

The norm from F to K is

$$N_{F/K}(a) = a^{2^m+1}.$$

It maps F^* onto K^* .

4.1.5 Frobenius and linearized polynomials

For $j \geq 0$, the map

$$\text{Frob}_j(a) = a^{2^j}$$

is $\mathbb{F}_2$ -linear. A linearized polynomial over F is a polynomial of the form

$$L(X) = \sum_{j=0}^{\lambda-1} c_j X^{2^j}, \quad c_j \in F.$$

It represents an $\mathbb{F}_2$ -linear map $F \rightarrow F$. All GALA maps $M_0, M_1, M_2, L_0, L_1, L_2, L_3$ are public invertible maps of this type.

In the QuickSilver model, applying a Frobenius map or any public linearized polynomial to an authenticated value does not increase its authentication degree. This is the reason that terms such as a^2 and a^{2^m} in the subfield-compression check are treated as linear views of a , not as extra field multiplications for the degree count.

4.1.6 Subfield coordinate extraction

Although K is a subfield of F , a subfield element is still stored as a λ -bit vector in the ambient polynomial basis. Only m coordinates are independent. The implementation fixes a deterministic extraction map

$$\text{Select}_P : K \rightarrow \{0, 1\}^m$$

and a deterministic embedding map

$$\text{Embed}_{\mathcal{B}'} : \{0, 1\}^m \rightarrow K \subset F.$$

They are generated from the field polynomial as follows:

1. Let $z_{\text{sub}} = \alpha^{(2^\lambda-1)/(2^m-1)}$. For the submitted field polynomials this element lies in K and the powers below have full $\mathbb{F}_2$ -rank m .
2. Form the basis $1, z_{\text{sub}}, \dots, z_{\text{sub}}^{m-1}$ and write each element as a λ -bit vector in the ambient polynomial basis.
3. Stack these vectors into an $m \times \lambda$ binary matrix and compute its reduced row echelon form.
4. The pivot columns define $P = (p_0, \dots, p_{m-1})$; the rows of the RREF define the embedding basis $\mathcal{B}' = (B'_0, \dots, B'_{m-1})$.

For $v \in K$,

$$\text{Select}_P(v) = (v_{p_0}, \dots, v_{p_{m-1}}),$$

and for $\sigma \in \{0, 1\}^m$,

$$\text{Embed}_{\mathcal{B}'}(\sigma) = \sum_{j=0}^{m-1} \sigma_j B'_j.$$

The two maps are inverse on K :

$$\text{Embed}_{\mathcal{B}'}(\text{Select}_P(v)) = v \quad \text{for all } v \in K.$$

Equivalently, the pivot columns of $\mathcal{B}'$ form the identity matrix, so

$$\text{Select}_P(\text{Embed}_{\mathcal{B}'}(\sigma)) = \sigma \quad \text{for all } \sigma \in \{0, 1\}^m.$$

Thus Select_P is used only to compress a subfield element to its m independent coordinates, while $\text{Embed}_{\mathcal{B}'}$ recovers the corresponding element of $K \subset F$ from those coordinates.

4.2 Symmetric Primitives

GALAS uses the hash and XOF interfaces provided by the NGCC auxiliary library. The specification treats these interfaces as domain-separated random-oracle and pseudorandom-expansion calls, following the abstraction used for VOLEitH signatures. The concrete implementation uses `pseudohash` for fixed-output hashing and `pseudoXOF` for variable-length expansion.

4.2.1 Byte Framing and Domain Separation

All byte strings absorbed by the hash/XOF layer use the serialization rules of Section 4.1.3: field elements are little-endian byte strings, integer counters and tweaks are little-endian fixed-width integers, and concatenation is performed in the order displayed below. No implicit ASCII string such as “Galas”, no instance name, and no variable-length separator is inserted by the submitted algorithms. Domain separation for fixed-output hash calls is provided by a final one-byte separator. The field lengths, parameter set, and the fact that a variable-length message appears only as the final field of $\mathbf{H}_{\text{msg}}$ make the framing unambiguous.

Table 5: Normative fixed-output hash calls. The separator byte is appended after the listed input fields and before the 32-bit counter used for counter-separated digest blocks.

Call	Sep.	Absorbed input fields, in order	Output bits	Split/use
$\mathbf{H}_{\text{msg}}$	0x08	$pk \in \{0, 1\}^{2\lambda}$, then message M	2λ	μ
$\mathbf{H}_{\text{seed}}$	0x03	$k \in \{0, 1\}^\lambda$, $\mu \in \{0, 1\}^{2\lambda}$	$\lambda + n_{\text{iv}}$	$K_{\text{root}} \parallel iv_{\text{pre}}$
$\mathbf{H}_{\text{iv}}$	0x04	$iv_{\text{pre}} \in \{0, 1\}^{n_{\text{iv}}}$	n_{iv}	iv
$\mathbf{H}_{\text{com}}$	0x01	BAVC commitment list, or BAVC tree-root list	2λ	BAVC root or subtree digest
$\mathbf{H}_{\text{FS1}}$	0x09	μ, h, c, iv	$5\lambda + 64$	$A_0 \parallel A_1 \parallel A_2 \parallel A_3 \parallel s_{\text{VOLE}} \parallel t_{\text{VOLE}}$
$\mathbf{H}_{\text{FS2}}$	0x0a	$chall_1, \pi_{\text{VOLE}}, T_{\text{VOLE}}, d$	$3\lambda + 64$	$R_0 \parallel R_1 \parallel s \parallel t$
$\mathbf{H}_{\text{FS3}}$	0x0b	$chall_2, \chi_{\text{QS}}, \pi_{\text{QS}}, \text{uint32_le}(ctr)$	λ	Δ

In Table 5, h is the 2λ -bit BAVC commitment root, c has length $(\tau - 1)\hat{\ell}$ bits, π_{VOLE} has length $\lambda + B$ bits, and T_{VOLE} has length $\lambda(\lambda + B)$ bits. The value $A_0, A_1, A_2, A_3, s_{\text{VOLE}}$ are field elements in F , and $t_{\text{VOLE}} \in \mathbb{F}_{2^{64}}$. Likewise, $R_0, R_1, s \in F$ and $t \in \mathbb{F}_{2^{64}}$.

4.2.2 Required Primitive Strength

The hash and XOF calls are not meant to raise the security level beyond the field size. They must instead avoid becoming the bottleneck for the claimed levels in Table 1. A single `pseudohash` call returns a raw digest block of the size shown in Table 6. A protocol hash request may be longer than one raw block; in that case the fixed-output wrapper uses the counter-separated construction of Section 4.2.3. If a protocol field requires fewer bits than a raw digest call returns, the result is truncated to the specified field length.

Table 6: Raw `pseudohash` block sizes and the largest fixed-output hash request used by the submitted algorithms. The largest request is $|chall_1| = 5\lambda + 64$ bits, not the 2λ -bit BAVC commitment length.

Instance family	Claimed target	Raw digest	Longest request	Blocks
Galas-160	128(80)	512 bits	864 bits	2
Galas-256	256(128)	512 bits	1344 bits	3
Galas-384	384(192)	768 bits	1984 bits	3
Galas-512	512(256)	1024 bits	2624 bits	3

For $\lambda = 160$, the raw NGCC `pseudohash` digest used by the wrapper is 512 bits. This does not mean that 512-bit security is claimed for the 160-bit field instance, and it also does not mean every fixed-output request fits in one raw digest block. For example, H_{FS1} requests $5\lambda + 64 = 864$ bits and H_{FS2} requests $3\lambda + 64 = 544$ bits at $\lambda = 160$; both are produced by two counter-separated `pseudohash` calls. Protocol values such as Δ , field elements, BAVC commitments, and VOLE masks are truncated or expanded exactly to their specified lengths.

The XOF/PRG interface is required to provide pseudorandom streams at the corresponding λ -bit field level. For seed-tree and VOLE expansion, the caller requests the exact number of output bits needed by the algorithm. The NGCC `pseudoXOF` interface is used as the submitted instantiation of this expansion layer. The submitted algorithms use $n_{iv} = 160$ -bit iv_{pre} and iv values in all parameter sets, matching the specified per-key usage bound of at most 2^{80} distinct signed messages.

4.2.3 Fixed-Output Hashing

The fixed-output wrapper is denoted by

$$H_D(\text{input}, n),$$

where D is a domain label and n is the requested number of output bits. Let X be the byte string obtained by concatenating the absorbed input fields for D in Table 5, and let sep_D be the corresponding one-byte separator. Let r_λ be the raw `pseudohash` digest size from Table 6. The wrapper computes

$$B_j = \text{pseudohash}(r_\lambda, X \parallel sep_D \parallel \text{uint32_le}(j)) \quad (j = 0, 1, 2, \dots)$$

and returns the first n bits of $B_0 \parallel B_1 \parallel B_2 \parallel \dots$. The last block is truncated if necessary. Thus the counter is always a 32-bit little-endian integer appended after the one-byte separator.

4.2.4 XOF and PRG Expansion

The XOF wrapper is denoted by

$$\text{XOF}_D(\text{seed}, \text{tweak}, n).$$

It calls NGCC `pseudoXOF` to obtain exactly n output bits from a fixed-width byte frame. The submitted implementation uses the following PRG frames, where `u32le` denotes a 32-bit little-endian encoding:

- GGM tree expansion and VOLE row expansion absorb the frame $seed \parallel iv \parallel \text{u32le}(tweak) \parallel \text{u32le}(counter)$.
- BAVC leaf commitment absorbs $seed \parallel iv \parallel \text{u32le}(tweak) \parallel 0\text{x}00$; it outputs 3λ bits, split as a λ -bit leaf seed and a 2λ -bit leaf commitment.

The seed, IV, tweak, and counter identify the GGM tree node, VOLE row, or leaf-expansion call. The algorithms in Section 6 use this interface only as a pseudorandom expansion function; no property of **pseudoXOF** outside this abstraction is required by the specification.

5 The GALA OWF and Constraint Relation

5.1 The GALA One-Way Function

5.1.1 Public constants

For each value of λ , the public parameters of GALA consist of:

- constants $c_0, c_1, c_2 \in F$;
- invertible $\mathbb{F}_2$ -linear maps $M_0, M_1, M_2 : F \rightarrow F$;
- invertible $\mathbb{F}_2$ -linear maps $L_0, L_1, L_2, L_3 : F \rightarrow F$.

Each map is represented as a linearized polynomial over F . The constants and linear maps are fixed per parameter set and are generated deterministically by the parameter-generation scripts in the submission. They are not secret and are not sampled during key generation or signing.

The normative constant values are fixed by the machine-readable constant tables identified in Appendix B. That appendix gives the submitted values of c_0, c_1, c_2 , the canonical serialization of the linear maps, and SHA-256 hashes of the files that contain the full tables. The reference and optimized implementations read the same serialized constants; if an implementation-specific representation differs internally, it must decode to the byte strings specified in Appendix B.

The generation procedure is included for reproducibility and follows the GALAS paper [LSP+26]: initialize a SHAKE256 stream with the ASCII label **Galas- λ** ; draw $\lambda/8$ bytes at a time; interpret each draw as a big-endian integer before mapping it to F ; take the first three field elements as c_0, c_1, c_2 ; then generate the coefficient vectors of $M_0, M_1, M_2, L_0, L_1, L_2, L_3$ as linearized polynomials. A candidate map is accepted only if its associated binary matrix is invertible and the linearized-polynomial coefficient vector has full support, meaning that every Frobenius coefficient a_j in $\sum_{j=0}^{\lambda-1} a_j X^{2^j}$ is nonzero. This condition rules out sparse public maps at parameter-generation time; it is not a secret-key condition. The generated values are encoded using the little-endian field-element encoding from Section 4.1.3. This SHAKE256 procedure is an offline parameter-generation procedure only; it is not called by KeyGen, Sign, Verify, or the NGCC hash/XOF wrapper.

5.1.2 Inversion convention

The nonlinear map is

$$S(a) = a^{-1} = a^{2^\lambda - 2} \quad \text{for } a \neq 0.$$

The implementation may define $S(0) = 0$ as a total function, but valid keys satisfy **Good**(k, x) and never use the zero case in a proof.

5.1.3 Evaluation

For $k, x \in F$, define

$$\begin{aligned} a_0 &= M_0(x \oplus k \oplus c_0), & a_1 &= M_1(k \oplus c_1), & a_2 &= M_2(k \oplus c_2), \\ b_0 &= S(a_0), & b_1 &= S(a_1), & b_2 &= S(a_2), \\ z &= L_0(b_0) \oplus L_1(b_1) \oplus L_2(b_2), \\ y &= S(z) \oplus L_3(k). \end{aligned}$$

The one-way function is $\text{GALA}_k(x) = y$.

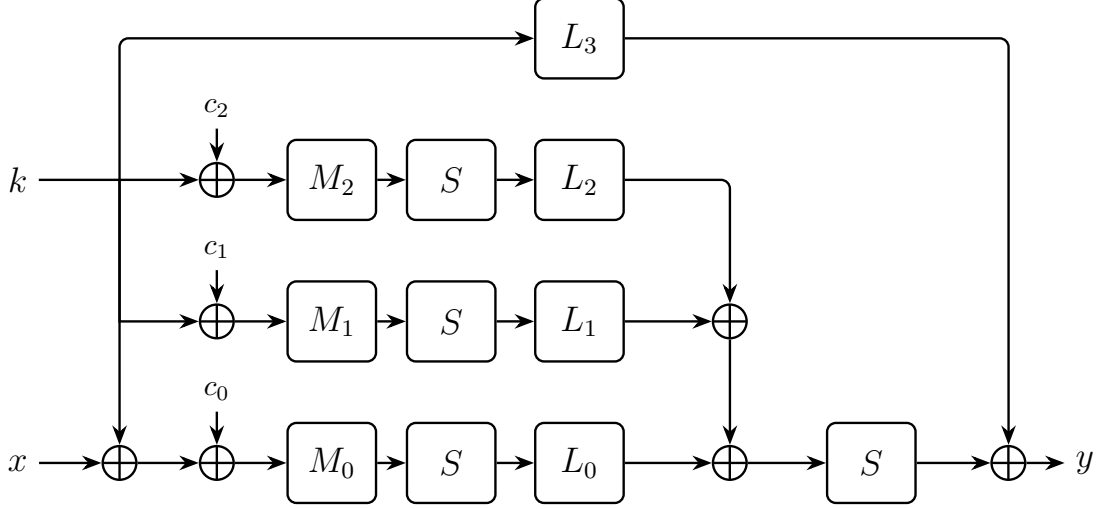


Figure 1: Wide-then-narrow structure of the GALA keyed OWF [LSP⁺26]. The three wide inversions are checked with subfield witnesses; the final inversion is tied to the public output.

Algorithm 1 $\text{GalaEval}(k, x)$

- 1: $a_0 \leftarrow M_0(x \oplus k \oplus c_0)$
 - 2: $a_1 \leftarrow M_1(k \oplus c_1)$
 - 3: $a_2 \leftarrow M_2(k \oplus c_2)$
 - 4: $b_0 \leftarrow S(a_0); \quad b_1 \leftarrow S(a_1); \quad b_2 \leftarrow S(a_2)$
 - 5: $z \leftarrow L_0(b_0) \oplus L_1(b_1) \oplus L_2(b_2)$
 - 6: $y \leftarrow S(z) \oplus L_3(k)$
 - 7: **return** y
-

5.1.4 Good predicate

The predicate $\text{Good}(k, x)$ is true if and only if all of the following hold:

1. the first two serialized bits of k are set: $k[0] = k[1] = 1$;
2. the three wide-round S-box inputs are nonzero: $a_0, a_1, a_2 \neq 0$;
3. the narrowed value before the final inversion is nonzero: $z \neq 0$.

Key generation rejection-samples (k, x) until $\text{Good}(k, x) = 1$.

Algorithm 2 Good(k, x)

```
1: if  $k[0] \neq 1$  or  $k[1] \neq 1$  then
2:   return 0
3:  $a_0 \leftarrow M_0(x \oplus k \oplus c_0)$ 
4:  $a_1 \leftarrow M_1(k \oplus c_1)$ 
5:  $a_2 \leftarrow M_2(k \oplus c_2)$ 
6: if  $a_0 = 0$  or  $a_1 = 0$  or  $a_2 = 0$  then
7:   return 0
8:  $b_0 \leftarrow S(a_0); b_1 \leftarrow S(a_1); b_2 \leftarrow S(a_2)$ 
9:  $z \leftarrow L_0(b_0) \oplus L_1(b_1) \oplus L_2(b_2)$ 
10: if  $z = 0$  then
11:   return 0
12: return 1
```

The bit condition is enforced both by key generation and by the QuickSilver constraint $k[0] \cdot k[1] + 1 = 0$ over characteristic two. The nonzero S-box conditions are enforced by key-generation rejection sampling and by honest witness generation. They define the valid public-key distribution and are needed for the subfield-compressed inverse witnesses.

5.2 Subfield Compression

This section specifies the compressed witnesses used for the three wide-round inversions in GALA.

5.2.1 Compressed inverse witness

Let $a \in F^*$ be an S-box input and let $m = \lambda/2$. Define

$$\nu(a) = N_{F/K}(a)^{-1} = (a^{2^m+1})^{-1} \in K.$$

The committed witness for this S-box is the m -bit string

$$\sigma = \text{Select}_P(\nu(a)).$$

When the verifier or prover needs the field element represented by σ , it uses

$$\bar{\sigma} = \text{Embed}_{B'}(\sigma) = \nu(a) \in K \subset F.$$

Only σ is committed as witness data. It has length $m = \lambda/2$ bits. The value $\bar{\sigma}$ is the embedded subfield element obtained by a public $\mathbb{F}_2$ -linear map; when it participates in field arithmetic it is represented as an ambient λ -bit element of F , but it is not an additional λ -bit witness.

Algorithm 3 InvNormWitness(a)

Require: $a \in F^*$

```
1:  $m \leftarrow \lambda/2$ 
2:  $t \leftarrow a^{2^m}$ 
3:  $u \leftarrow a \cdot t$ 
4:  $v \leftarrow u^{-1}$ 
5:  $\sigma \leftarrow \text{Select}_P(v)$ 
6: return  $\sigma$ 
```

$\triangleright$ Frobenius map
 $\triangleright u = N_{F/K}(a) \in K$
 $\triangleright v \in K$

5.2.2 Reconstructing the inverse

The compressed witness determines the inverse of a by

$$b = \bar{\sigma} \cdot a^{2^m}.$$

Indeed, if $\bar{\sigma} = (a^{2^m+1})^{-1}$, then

$$\bar{\sigma} \cdot a^{2^m} = (a^{2^m+1})^{-1} \cdot a^{2^m} = a^{-1}.$$

Thus the signer never commits to the full λ -bit inverse b ; it commits only to the m -bit coordinate string $\sigma \in \{0, 1\}^m$.

5.2.3 Consistency identity

The correctness of the compressed witness is checked by the identity

$$\bar{\sigma} \cdot a^{2^m} \cdot a = 1.$$

Substituting $\bar{\sigma} = (a^{2^m+1})^{-1}$ gives

$$(a^{2^m+1})^{-1} \cdot a^{2^m} \cdot a = 1.$$

Equivalently, if $b = \bar{\sigma} \cdot a^{2^m}$, then the checked identity is $b \cdot a = 1$. Multiplying by a gives $b \cdot a^2 = a$, the reconstruction identity used above.

In the QuickSilver degree accounting, a and a^{2^m} are both degree-1 authenticated quantities because Frobenius maps are $\mathbb{F}_2$ -linear. The embedded witness $\bar{\sigma}$ is also degree 1. Therefore the product $\bar{\sigma} \cdot a^{2^m} \cdot a$ has authentication degree 3.

5.2.4 Extended witness layout

For GALAS, the extended witness is

$$w = (k, \sigma_0, \sigma_1, \sigma_2) \in \{0, 1\}^\ell, \quad \ell = \lambda + 3m = \frac{5}{2}\lambda.$$

The layout is fixed as follows:

Offset	Length	Content
0	λ	secret key k
λ	m	$\sigma_0 = \text{InvNormWitness}(a_0)$
$\lambda + m$	m	$\sigma_1 = \text{InvNormWitness}(a_1)$
$\lambda + 2m$	m	$\sigma_2 = \text{InvNormWitness}(a_2)$

Compared with committing to the secret key and three full inverse outputs, which would require 4λ bits, this layout reduces the committed witness length by $3\lambda/2$ bits, or 37.5%.

5.3 Degree-3 Constraint System

This section specifies the QuickSilver constraints proving that the committed witness $w = (k, \sigma_0, \sigma_1, \sigma_2)$ satisfies $y = \text{GALA}_k(x)$.

5.3.1 Authenticated values and degree

QuickSilver evaluates expressions on authenticated field values at a hidden point $\Delta \in F$. A committed witness bit is treated as an authenticated degree-1 value. Public constants have degree 0. The authentication degree obeys the following rules:

- addition and multiplication by public constants do not increase degree;
- public $\mathbb{F}_2$ -linear maps, including Frobenius maps, do not increase degree;
- multiplying authenticated expressions adds their degrees.

All constraints in GALAS have degree at most 3.

5.3.2 Loading the witness

The first λ witness bits are interpreted as the secret key $k \in F$. Each m -bit string σ_i is embedded as

$$\bar{\sigma}_i = \text{Embed}_{\mathcal{B}'}(\sigma_i) \in K \subset F.$$

The notation distinguishes the committed coordinate string σ_i from the derived field element $\bar{\sigma}_i$: σ_i contributes only m witness bits, whereas $\bar{\sigma}_i$ is the public linear image used in the polynomial constraints. Since embedding is an $\mathbb{F}_2$ -linear map from bits to F , each $\bar{\sigma}_i$ is an authenticated degree-1 value.

The public key provides $x, y \in F$. Define authenticated values

$$\begin{aligned} a_0 &= M_0(x \oplus k \oplus c_0), \\ a_1 &= M_1(k \oplus c_1), \\ a_2 &= M_2(k \oplus c_2). \end{aligned}$$

Each a_i has degree 1.

5.3.3 Constraint C1: restricted key space

Let $k[0]$ and $k[1]$ be the first two committed key bits. The constraint

$$P_{\text{key}} = k[0] \cdot k[1] + 1 = 0$$

forces $k[0] = k[1] = 1$ over characteristic two. This constraint has degree 2.

5.3.4 Constraint C2: compressed wide inversions

For each $i \in \{0, 1, 2\}$, impose

$$P_{\text{norm},i} = \bar{\sigma}_i \cdot a_i^{2^m} \cdot a_i + 1 = 0.$$

The term $a_i^{2^m}$ is a Frobenius image of a_i , hence a degree-1 authenticated value. The product contains three degree-1 quantities, so $P_{\text{norm},i}$ has degree 3.

The constraint forces $a_i \neq 0$ and gives $\bar{\sigma}_i \cdot a_i^{2^m} \cdot a_i = 1$. Therefore $\bar{\sigma}_i = (a_i^{2^m+1})^{-1}$ and the corresponding inverse is

$$b_i = \bar{\sigma}_i \cdot a_i^{2^m} = a_i^{-1}.$$

The reconstructed values b_i have authentication degree 2. Multiplying $b_i a_i = 1$ by a_i also gives $b_i a_i^2 = a_i$, which is the equivalent identity used to justify the inverse reconstruction for valid inputs.

5.3.5 Constraint C3: final inverse tied to the public output

Using the reconstructed wide inverse outputs, define

$$z = L_0(b_0) \oplus L_1(b_1) \oplus L_2(b_2).$$

Because the L_i are public $\mathbb{F}_2$ -linear maps, z has degree 2. Define

$$t = y \oplus L_3(k),$$

which has degree 1 because y is public and L_3 is linear. The final constraint is

$$P_{\text{out}} = z \cdot t + 1 = 0.$$

It has degree 3. When it holds, $t = z^{-1}$, hence $y = z^{-1} \oplus L_3(k)$ and the public output matches the GALA evaluation.

5.3.6 Constraint summary

Name	Constraint	Degree	Count
C1	$k[0] \cdot k[1] + 1 = 0$	2	1
C2	$\bar{\sigma}_i a_i^{2^m} a_i + 1 = 0$	3	3
C3	$z(y \oplus L_3(k)) + 1 = 0$	3	1
Total		≤ 3	5

5.3.7 Complete OWF constraint algorithm

Algorithm 4 is the GALAS analogue of the FAEST ZK.OWFConstraints procedure. It is written once and used in both prover and verifier logic. On the prover side, the bracketed values are authenticated polynomials in the QuickSilver indeterminate X . On the verifier side, the same operations are evaluated at $X = \Delta$ after the verifier has reconstructed the authenticated witness values. In the algorithm, $\langle \sigma_i \rangle$ denotes the authenticated m -coordinate witness, while $\langle \bar{\sigma}_i \rangle$ denotes its embedded subfield element after applying **Embed** _{$\mathcal{B}'$} componentwise.

Promotion to degree 3. The notation **Promote**₃(E) is not a separate cryptographic subroutine. It is the bookkeeping operation that views a QuickSilver authenticated expression $E(X)$ of degree at most 3 in the common degree-3 coefficient format

$$E(X) = e_0 + e_1X + e_2X^2 + e_{\text{val}}X^3.$$

The last coefficient is the plaintext constraint value. For a valid witness, $e_{\text{val}} = 0$; the masked coefficients e_0, e_1, e_2 are the values that enter the **ZKHash** computation. Missing high mask coefficients are set to zero, so a linear or quadratic constraint can be batched together with cubic constraints by the same interface. In verifier mode, where only the evaluation point $X = \Delta$ is available, **Promote**₃ leaves the evaluated value unchanged; the name only records that the expression is checked under the degree-3 QuickSilver mask layout.

Algorithm 4 Gala.OWFConstraints($\langle w \rangle, pk$)

```
1: Parse  $pk = x \parallel y$  and  $\langle w \rangle = (\langle k \rangle, \langle \sigma_0 \rangle, \langle \sigma_1 \rangle, \langle \sigma_2 \rangle)$ 
2: for  $i \in \{0, 1, 2\}$  do
3:    $\langle \bar{\sigma}_i \rangle \leftarrow \text{Embed}_{\mathcal{B}'}(\langle \sigma_i \rangle)$ 
4:    $\langle a_0 \rangle \leftarrow M_0(x \oplus \langle k \rangle \oplus c_0)$ 
5:    $\langle a_1 \rangle \leftarrow M_1(\langle k \rangle \oplus c_1)$ 
6:    $\langle a_2 \rangle \leftarrow M_2(\langle k \rangle \oplus c_2)$ 
7:    $C_0 \leftarrow \text{Promote}_3(\langle k[0] \rangle \cdot \langle k[1] \rangle \oplus 1)$ 
8:   for  $i \in \{0, 1, 2\}$  do
9:      $E_i \leftarrow \langle \bar{\sigma}_i \rangle \cdot \langle a_i \rangle^{2^m} \cdot \langle a_i \rangle \oplus 1$ 
10:     $C_{i+1} \leftarrow \text{Promote}_3(E_i)$ 
11:     $\langle b_i \rangle \leftarrow \langle \bar{\sigma}_i \rangle \cdot \langle a_i \rangle^{2^m}$ 
12:    $\langle z \rangle \leftarrow L_0(\langle b_0 \rangle) \oplus L_1(\langle b_1 \rangle) \oplus L_2(\langle b_2 \rangle)$ 
13:    $\langle t \rangle \leftarrow y \oplus L_3(\langle k \rangle)$ 
14:    $C_4 \leftarrow \text{Promote}_3(\langle z \rangle \cdot \langle t \rangle \oplus 1)$ 
15: return  $(C_0, C_1, C_2, C_3, C_4)$ 
```

The algorithm intentionally contains no branch on $a_i = 0$. The nonzero conditions are part of the public-key distribution enforced by Algorithm 19; the proof relation checks the displayed polynomials for keys generated by that distribution. All Frobenius powers and all maps M_i, L_i are public $\mathbb{F}_2$ -linear operations and are therefore applied componentwise to authenticated values without increasing QuickSilver degree.

5.3.8 Batching into QuickSilver responses

The five constraint polynomials returned by Algorithm 4 as $(C_0, C_1, C_2, C_3, C_4)$ correspond respectively to the named constraints C1, C2₀, C2₁, C2₂, and C3 in the summary table above. They are fed in that order into the QuickSilver universal hash. This hash is the random linear combination used by the protocol; there is no separate list of batching coefficients in the serialized transcript. For the prover, each promoted constraint is represented in the degree-3 QuickSilver format

$$C_j(X) = a_{j,0} + a_{j,1}X + a_{j,2}X^2 + a_{j,\text{val}}X^3.$$

For an honest witness, $a_{j,\text{val}} = 0$ for all j . Let

$$A_i = (a_{0,i}, a_{1,i}, a_{2,i}, a_{3,i}, a_{4,i}) \quad (i = 0, 1, 2)$$

be the ordered coefficient vectors. The two λ -bit auxiliary VOLE blocks after the witness rows mask the three QuickSilver hash outputs. The two serialized response elements are called θ_1 and θ_2 , while the non-serialized QuickSilver check value is called χ_{QS} . The verifier recomputes χ_{QS} and binds it into $\mathbf{H}_{\text{FS3}}$.

For a length- λ bit block $U[a..a + \lambda)$ and a tag matrix T , define

$$\begin{aligned} \text{Val}(U, a) &= \sum_{r=0}^{\lambda-1} U[a+r] \alpha^r \in F, \\ \text{Tag}(T, a) &= \sum_{r=0}^{\lambda-1} T[a+r, *] \alpha^r \in F. \end{aligned}$$

Here $T[a+r, *]$ denotes row $a+r$ of T , interpreted as a field element. The implementation uses $a = \ell$ and $a = \ell + \lambda$ for the two auxiliary blocks.

The Low_{64} and ι_{64} maps used below are the coordinate maps defined in Section 4.1.2. The 64-bit universal-hash branch uses Low_{64} for each input element and then applies ι_{64} only at finalization.

Algorithm 5 $\text{ZKHash}_{\text{chall}_2}(Z, M)$

- 1: Parse $\text{chall}_2 = R_0 \parallel R_1 \parallel s \parallel t$, where $R_0, R_1, s \in F$ and $t \in \mathbb{F}_{2^{64}}$
 - 2: $H_s \leftarrow 0$; $H_t \leftarrow 0$
 - 3: **for** each field element z in the ordered list Z **do**
 - 4: $H_s \leftarrow sH_s \oplus z$
 - 5: $H_t \leftarrow tH_t \oplus \text{Low}_{64}(z)$
 - 6: **return** $M \oplus R_0H_s \oplus R_1\iota_{64}(H_t)$
-

In the second Horner chain, multiplication by the 64-bit key t is performed in $\mathbb{F}_{2^{64}}$ using the polynomial $X^{64} + X^4 + X^3 + X + 1$. The state is zero-extended to F only when it is multiplied by the public coefficient R_1 at the end. This is the QuickSilver **ZKHash** from FAEST, specialized to the five GALA constraints.

The following algorithms specify the logical QuickSilver interface used by Section 7. They abstract over the byte-level universal hash implementation but fix the algebraic data being checked.

Algorithm 6 $\text{QS.Prove}(\text{chall}_2, w, u, V, pk)$

- 1: Parse $pk = x \parallel y$ and $w = (k, \sigma_0, \sigma_1, \sigma_2)$
 - 2: Let $u_{\text{QS}} = u[0..\ell + 2\lambda)$ and $V_{\text{QS}} = V[0..\ell + 2\lambda, *]$; ignore the final $\lambda + B$ VOLEHash mask rows
 - 3: Split u_{QS} and V_{QS} into witness rows and two auxiliary λ -row blocks at offsets ℓ and $\ell + \lambda$
 - 4: Form authenticated polynomials $P_k(X)$ and $P_{\sigma_i}(X)$ from the witness bits and their tags
 - 5: Let $\langle w \rangle = (P_k, P_{\sigma_0}, P_{\sigma_1}, P_{\sigma_2})$
 - 6: $(C_0(X), \dots, C_4(X)) \leftarrow \text{Gala.OWFConstraints}(\langle w \rangle, pk)$
 - 7: **if** any value coefficient $a_{j,\text{val}}$ of C_j is nonzero **then**
 - 8: **return** $\perp$
 - 9: Let A_0, A_1, A_2 be the three ordered coefficient vectors
 - 10: $m_0 \leftarrow \text{Tag}(V_{\text{QS}}, \ell)$
 - 11: $m_1 \leftarrow \text{Val}(u_{\text{QS}}, \ell) \oplus \text{Tag}(V_{\text{QS}}, \ell + \lambda)$
 - 12: $m_2 \leftarrow \text{Val}(u_{\text{QS}}, \ell + \lambda)$
 - 13: $\chi_{\text{QS}} \leftarrow \text{ZKHash}_{\text{chall}_2}(A_0, m_0)$
 - 14: $\theta_1 \leftarrow \text{ZKHash}_{\text{chall}_2}(A_1, m_1)$
 - 15: $\theta_2 \leftarrow \text{ZKHash}_{\text{chall}_2}(A_2, m_2)$
 - 16: $\pi_{\text{QS}} \leftarrow \theta_1 \parallel \theta_2$
 - 17: **return** $(\pi_{\text{QS}}, \chi_{\text{QS}})$
-

Algorithm 6 takes w explicitly because it is the logical prover input. Implementations may realize the same interface by copying the witness bits into the first ℓ rows of the VOLE buffer before invoking the QuickSilver engine. The final $\lambda + B$ VOLEHash rows are deliberately not used by this algorithm. The three masks m_0, m_1, m_2 are the standard degree-3 QuickSilver coefficient masks: the first auxiliary block masks the constant coefficient hash, the second block and the first auxiliary value mask the linear coefficient hash, and the second auxiliary value masks the quadratic coefficient hash. Only the latter two response elements are serialized as π_{QS} .

Algorithm 7 $\text{QS.Verify}(chall_2, pk, d, Q, \Delta, \pi_{\text{QS}})$

- 1: Parse $pk = x \parallel y$ and split d according to the witness layout
 - 2: Let $Q_{\text{QS}} = Q[0..\ell + 2\lambda, *]$; ignore the final $\lambda + B$ VOLEHash mask rows
 - 3: Parse π_{QS} as $\theta_1 \parallel \theta_2$; if parsing fails, return $\perp$
 - 4: Recover authenticated values at $X = \Delta$: $\hat{w}_j(\Delta) = Q_{\text{QS}}[j, *] \oplus d_j \Delta$ for all witness bits
 - 5: Let $\langle w \rangle_\Delta$ denote these recovered authenticated witness values
 - 6: $(\hat{C}_0(\Delta), \dots, \hat{C}_4(\Delta)) \leftarrow \text{Gala.OWFConstraints}(\langle w \rangle_\Delta, pk)$
 - 7: $M \leftarrow \text{Tag}(Q_{\text{QS}}, \ell) \oplus (\theta_1 \oplus \text{Tag}(Q_{\text{QS}}, \ell + \lambda))\Delta \oplus \theta_2 \Delta^2$
 - 8: $\chi_{\text{QS}} \leftarrow \text{ZKHash}_{chall_2}((\hat{C}_0(\Delta), \dots, \hat{C}_4(\Delta)), M)$
 - 9: **return** χ_{QS}
-

Algorithm 7 never learns w or u . The value d and the row view of Q allow it to evaluate the authenticated witness at the single point Δ . It then runs the same GALAS constraint algorithm at that point. The mask expression

$$\text{Tag}(Q_{\text{QS}}, \ell) \oplus (\theta_1 \oplus \text{Tag}(Q_{\text{QS}}, \ell + \lambda))\Delta \oplus \theta_2 \Delta^2$$

is the verifier-side reconstruction of the prover's hidden χ_{QS} value. The final Fiat–Shamir hash in Algorithm 22 accepts only if this reconstructed value leads back to the serialized challenge Δ .

6 VOLEitH and QuickSilver Construction

This section specifies the proof machinery used by GALAS. The interfaces follow the VOLEitH and QuickSilver architecture: a BAVC commits to seed-tree leaves, the leaves are expanded into a VOLE correlation, VOLEHash checks consistency of the correlation, and QuickSilver checks the low-degree GALA relation at the Fiat–Shamir challenge point.

6.1 Batch All-But-One Vector Commitment (BAVC)

The batch all-but-one vector commitment (BAVC) used here is a one-tree seed-tree commitment built from the GGM paradigm and the one-tree optimization used in recent VOLEitH signatures [GGM84, BBdSG⁺23, BBM⁺24]. It commits to τ logical VOLE vectors inside one binary seed tree. For logical vector i , the opening challenge selects one hidden leaf index $i_\Delta[i]$. The verifier receives the commitment of the hidden leaf and enough sibling seeds to reconstruct every other leaf.

Let

$$k_i = \begin{cases} k_{tree}, & i < \tau_1, \\ k_{tree} - 1, & i \geq \tau_1, \end{cases} \quad N_i = 2^{k_i}, \quad \text{off}_i = \sum_{r=0}^{i-1} N_r.$$

The one-tree heap has $L = \sum_i N_i$ leaves and $2L - 1$ nodes. Node 0 is the root, the children of an internal node α are $2\alpha + 1$ and $2\alpha + 2$, and the leaves have heap positions $L - 1, \dots, 2L - 2$. The logical leaf (i, j) is placed at

$$\text{pos}(i, j) = \begin{cases} L - 1 + \tau j + i, & j < 2^{k_{tree}-1}, \\ L - 1 + \tau 2^{k_{tree}-1} + \tau_1(j \bmod 2^{k_{tree}-1}) + i, & j \geq 2^{k_{tree}-1}. \end{cases}$$

The second case occurs only for $i < \tau_1$.

The following helper expansions are normative. For a tree node seed $K_\alpha \in \{0, 1\}^\lambda$, the expansion $\text{TreeExpand}(K_\alpha, iv, \alpha)$ produces $K_{2\alpha+1} \parallel K_{2\alpha+2}$. It uses the PRG frame

$$K_\alpha \parallel iv \parallel \text{u32le}(\alpha) \parallel \text{u32le}(ctr).$$

Output is generated in 128-bit blocks. If $b = \lceil 2\lambda/128 \rceil$ blocks are required, the first call uses $ctr = 0$ and emits $\min(b, 4 - (b \bmod 2))$ blocks; later calls emit two blocks with ctr equal to the number of blocks already emitted. The first 2λ bits are split into the two child seeds. For a leaf node seed $K_{\text{pos}(i,j)}$, define

$$sd_{i,j} \parallel com_{i,j} = \text{LeafExpand}(K_{\text{pos}(i,j)}, iv, L - 1 + i),$$

where **LeafExpand** calls **pseudoX0F** on $K_{\text{pos}(i,j)} \parallel iv \parallel \text{u32le}(L - 1 + i) \parallel 0\text{x}00$ and returns 3λ bits, split into a λ -bit VOLE leaf seed $sd_{i,j}$ and a 2λ -bit leaf commitment $com_{i,j}$.

For a list of leaf commitments, define

$$h_i = H_{\text{com}}(com_{i,0} \parallel \dots \parallel com_{i,N_i-1}), \quad h = H_{\text{com}}(h_0 \parallel \dots \parallel h_{\tau-1}),$$

where H_{com} is the fixed-output hash call in Table 5. Each h_i and h has length 2λ bits.

Algorithm 8 BAVC.Commit(K_{root}, iv)

```

1: Set  $K_0 \leftarrow K_{\text{root}}$ 
2: for  $\alpha = 0$  to  $L - 2$  do
3:    $K_{2\alpha+1} \parallel K_{2\alpha+2} \leftarrow \text{TreeExpand}(K_\alpha, iv, \alpha)$ 
4: for  $i = 0$  to  $\tau - 1$  do
5:   for  $j = 0$  to  $N_i - 1$  do
6:      $(sd_{i,j}, com_{i,j}) \leftarrow \text{LeafExpand}(K_{\text{pos}(i,j)}, iv, L - 1 + i)$ 
7:    $h_i \leftarrow H_{\text{com}}(com_{i,0} \parallel \dots \parallel com_{i,N_i-1})$ 
8:  $h \leftarrow H_{\text{com}}(h_0 \parallel \dots \parallel h_{\tau-1})$ 
9: Store  $K_\alpha$ ,  $sd_{i,j}$ , and  $com_{i,j}$  in  $st_{\text{BAVC}}$ 
10: return  $(h, st_{\text{BAVC}})$ 
```

Algorithm 9 BAVC.Open($st_{\text{BAVC}}, I_\Delta$)

```

1: Parse  $I_\Delta = (i_\Delta[0], \dots, i_\Delta[\tau - 1])$ 
2: if any  $i_\Delta[i] \notin [0, N_i)$  then
3:   return  $\perp$ 
4: Initialize a bitset hidden on the  $2L - 1$  tree nodes to 0
5: for  $i = 0$  to  $\tau - 1$  do
6:    $\alpha \leftarrow \text{pos}(i, i_\Delta[i])$ ; mark  $\alpha$  hidden
7:   while  $\alpha > 0$  and  $\lfloor (\alpha - 1)/2 \rfloor$  is not yet marked hidden do
8:      $\alpha \leftarrow \lfloor (\alpha - 1)/2 \rfloor$ ; mark  $\alpha$  hidden
9:  $Seeds \leftarrow []$ 
10: for  $\alpha = L - 2$  down to 0 do
11:    $left \leftarrow 2\alpha + 1$ ;  $right \leftarrow 2\alpha + 2$ 
12:    $hl \leftarrow \text{hidden}[left]$ ;  $hr \leftarrow \text{hidden}[right]$ 
13:   Set  $\text{hidden}[\alpha] \leftarrow hl \vee hr$ 
14:   if  $hl \oplus hr = 1$  then
15:     Append  $K_{left+hl}$  to  $Seeds$ 
16: if  $|Seeds| > T_{\text{open}}$  then
17:   return  $\perp$ 
18: Pad  $Seeds$  with all-zero  $\lambda$ -bit strings until  $|Seeds| = T_{\text{open}}$ 
19: return  $\pi_{\text{BAVC}} = com_{0,i_\Delta[0]} \parallel \dots \parallel com_{\tau-1,i_\Delta[\tau-1]} \parallel Seeds[0] \parallel \dots \parallel Seeds[T_{\text{open}} - 1]$ 
```

Algorithm 10 BAVC.Reconstruct($iv, I_\Delta, \pi_{\text{BAVC}}$)

```
1: Parse  $\pi_{\text{BAVC}}$  as  $C_0, \dots, C_{\tau-1}$ , each  $2\lambda$  bits, followed by  $S_0, \dots, S_{T_{\text{open}}-1}$ , each  $\lambda$  bits
2: if parsing fails, or any  $i_\Delta[i] \notin [0, N_i]$  then
3:   return  $\perp$ 
4: Initialize bitsets hidden and known on the  $2L - 1$  tree nodes to 0; set  $s \leftarrow 0$ 
5: for  $i = 0$  to  $\tau - 1$  do
6:   Mark  $\text{pos}(i, i_\Delta[i])$  hidden
7:   for  $\alpha = L - 2$  down to  $0$  do
8:      $left \leftarrow 2\alpha + 1$ ;  $right \leftarrow 2\alpha + 2$ 
9:      $hl \leftarrow \text{hidden}[left]$ ;  $hr \leftarrow \text{hidden}[right]$ 
10:    Set  $\text{hidden}[\alpha] \leftarrow hl \vee hr$ 
11:    if  $hl \oplus hr = 1$  then
12:      if  $s = T_{\text{open}}$  then
13:        return  $\perp$ 
14:      Set  $K_{left+hl} \leftarrow S_s$  and mark  $left + hl$  known
15:       $s \leftarrow s + 1$ 
16: if any byte of  $S_s, \dots, S_{T_{\text{open}}-1}$  is nonzero then
17:   return  $\perp$ 
18: for  $\alpha = 0$  to  $L - 2$  do
19:   if  $\alpha$  is known then
20:      $K_{2\alpha+1} \parallel K_{2\alpha+2} \leftarrow \text{TreeExpand}(K_\alpha, iv, \alpha)$ 
21:     Mark  $2\alpha + 1$  and  $2\alpha + 2$  known
22: for  $i = 0$  to  $\tau - 1$  do
23:   for  $j = 0$  to  $N_i - 1$  do
24:     if  $j = i_\Delta[i]$  then
25:        $com_{i,j} \leftarrow C_i$ 
26:     else
27:       if  $\text{pos}(i, j)$  is not known then
28:         return  $\perp$ 
29:        $(sd_{i,j}, com_{i,j}) \leftarrow \text{LeafExpand}(K_{\text{pos}(i,j)}, iv, L - 1 + i)$ 
30:       Append  $sd_{i,j}$  to  $st_{\text{rec}}$  in increasing  $(i, j)$  order
31:    $h_i \leftarrow H_{\text{com}}(com_{i,0} \parallel \dots \parallel com_{i,N_i-1})$ 
32:  $h \leftarrow H_{\text{com}}(h_0 \parallel \dots \parallel h_{\tau-1})$ 
33: return  $(h, st_{\text{rec}})$ 
```

6.2 VOLE Commitment and Reconstruction

The committed VOLE vector has length $\hat{\ell}$ bits. The first ℓ rows mask the GALA witness, the next 2λ rows mask the degree-3 QuickSilver coefficients, and the final $\lambda + B$ rows are used by the VOLE consistency hash. The prover obtains a vector $u \in \{0, 1\}^{\hat{\ell}}$ and tag matrix $V \in \{0, 1\}^{\hat{\ell} \times \lambda}$. Row r of V is the λ -bit field tag attached to VOLE coordinate r , while column j is the $\hat{\ell}$ -bit string used by VOLEHash. The verifier reconstructs the corresponding matrix $Q = V \oplus \Delta u$ for the final challenge $\Delta \in F$.

Define $\text{VoleExpand}(sd, iv, i)$ as follows: concatenate output blocks generated by pseudoXOF on the frame

$$sd \parallel iv \parallel \text{u32le}(2^{31} + i) \parallel \text{u32le}(ctr), \quad ctr = 0, 1, 2, \dots,$$

and return the first $\hat{\ell}$ bits. The small-VOLE folding algorithm below is the FAEST sender transform.

Algorithm 11 ConvertToVOLESender($i, sd_{i,0}, \dots, sd_{i,N_i-1}, iv$)

```
1: for  $j = 0$  to  $N_i - 1$  do
2:    $R_{0,j} \leftarrow \text{VoleExpand}(sd_{i,j}, iv, i)$ 
3: for  $d = 0$  to  $k_i - 1$  do
4:    $v_{i,d} \leftarrow 0^{\hat{\ell}}$ 
5:   for  $j = 0$  to  $2^{k_i-d-1} - 1$  do
6:      $v_{i,d} \leftarrow v_{i,d} \oplus R_{d,2j+1}$ 
7:      $R_{d+1,j} \leftarrow R_{d,2j} \oplus R_{d,2j+1}$ 
8:  $u_i \leftarrow R_{k_i,0}$ 
9: return  $(u_i, v_{i,0}, \dots, v_{i,k_i-1})$ 
```

Algorithm 12 ConvertToVOLEReceiver($i, i_\Delta[i], st_{\text{rec}}, iv$)

```
1: Set  $R_{0,0} \leftarrow 0^{\hat{\ell}}$ 
2: for  $j = 0$  to  $N_i - 1$  with  $j \neq i_\Delta[i]$  do
3:   Read  $sd_{i,j}$  from  $st_{\text{rec}}$ 
4:    $R_{0,j \oplus i_\Delta[i]} \leftarrow \text{VoleExpand}(sd_{i,j}, iv, i)$ 
5: for  $d = 0$  to  $k_i - 1$  do
6:    $q'_{i,d} \leftarrow 0^{\hat{\ell}}$ 
7:   for  $j = 0$  to  $2^{k_i-d-1} - 1$  do
8:      $q'_{i,d} \leftarrow q'_{i,d} \oplus R_{d,2j+1}$ 
9:      $R_{d+1,j} \leftarrow R_{d,2j} \oplus R_{d,2j+1}$ 
10: return  $(q'_{i,0}, \dots, q'_{i,k_i-1})$ 
```

Algorithm 13 VOLE.Commit(K_{root}, iv)

```
1:  $(h, st_{\text{BAVC}}) \leftarrow \text{BAVC.Commit}(K_{\text{root}}, iv)$ 
2: for  $i = 0$  to  $\tau - 1$  do
3:    $(u_i, v_{i,0}, \dots, v_{i,k_i-1}) \leftarrow \text{ConvertToVOLESender}(i, sd_{i,0}, \dots, sd_{i,N_i-1}, iv)$ 
4:  $u \leftarrow u_0$ 
5: for  $i = 1$  to  $\tau - 1$  do
6:    $c^i \leftarrow u_0 \oplus u_i$ 
7: Form the columns of  $V$  as  $(v_{0,0}, \dots, v_{0,k_0-1}, v_{1,0}, \dots, v_{\tau-1,k_{\tau-1}-1})$ 
8: Append  $w_{\text{grind}}$  all-zero columns to  $V$ , so that  $V$  has  $\lambda$  columns
9:  $c \leftarrow c^1 \parallel \dots \parallel c^{\tau-1}$ 
10: return  $(h, c, u, V, st_{\text{BAVC}})$ 
```

Algorithm 14 $\text{VOLE.Reconstruct}(iv, I_\Delta, \pi_{\text{BAVC}}, c)$

```
1:  $(h, st_{\text{rec}}) \leftarrow \text{BAVC.Reconstruct}(iv, I_\Delta, \pi_{\text{BAVC}})$ 
2: if  $h = \perp$  then
3:   return  $\perp$ 
4: Parse  $c = c^1 \parallel \dots \parallel c^{\tau-1}$ , each  $\hat{\ell}$  bits
5:  $Q \leftarrow []$ 
6: for  $i = 0$  to  $\tau - 1$  do
7:    $(q'_{i,0}, \dots, q'_{i,k_i-1}) \leftarrow \text{ConvertToVoleReceiver}(i, i_\Delta[i], st_{\text{rec}}, iv)$ 
8:   for  $d = 0$  to  $k_i - 1$  do
9:     if  $i = 0$  then
10:      Append  $q'_{i,d}$  to  $Q$ 
11:     else
12:      Append  $q'_{i,d} \oplus ((i_\Delta[i] \text{ has bit } d) \cdot c^i)$  to  $Q$ 
13: Append  $w_{\text{grind}}$  all-zero columns to  $Q$ , so that  $Q$  has  $\lambda$  columns
14: return  $(h, Q)$ 
```

6.3 VOLE Consistency Hash

VOLEHash binds the prover's VOLE vector and tag matrix to the transcript. It is a universal-hash check with output length $\lambda + B$ bits, where $B = 16$ bits. The sender computes a serialized proof value π_{VOLE} from (u, V) and the first Fiat-Shamir challenge. The receiver uses (Q, Δ) and the same π_{VOLE} to absorb the same value into the second challenge transcript. The low-level universal hash uses a λ -bit branch over $F = \mathbb{F}_{2^\lambda}$ and a 64-bit branch over the auxiliary field $\mathbb{F}_{2^{64}}$ of Section 4.1.2. The non-serialized value T_{VOLE} is an ordered byte string absorbed into $\mathbf{H}_{\text{FS2}}$: it consists of λ blocks of length $\lambda + B$ bits, one for each VOLE tag column. Hence $|T_{\text{VOLE}}| = \lambda(\lambda + B)$ bits, while $|\pi_{\text{VOLE}}| = \lambda + B$ bits.

Let

$$R_{\text{VOLE}} = \ell + 2\lambda, \quad \eta = \lambda + B, \quad n_\lambda = \lceil R_{\text{VOLE}}/\lambda \rceil, \quad n_{64} = \lceil \lambda/64 \rceil.$$

For a byte string $X \in \{0, 1\}^{\hat{\ell}}$, define $\text{Tail}(X) = X[R_{\text{VOLE}}..R_{\text{VOLE}} + \eta)$. These are exactly the last $\lambda + B$ VOLEHash mask rows of a VOLE vector. For $0 \leq r < n_\lambda$, let $C_r(X) \in F$ be the λ -bit little-endian field element obtained from the block $X[r\lambda.. \min((r+1)\lambda, R_{\text{VOLE}}))$, padded with zeros to λ bits if necessary. For $0 \leq j < n_{64}$, let $W_{r,j}(X) \in \mathbb{F}_{2^{64}}$ be the j -th 64-bit little-endian word of the serialized $C_r(X)$, padded with zeros if the word is partial.

Algorithm 15 $\text{VOLEColumnHash}_{\text{chall}_1}(X)$

```
1: Parse  $\text{chall}_1 = A_0 \parallel A_1 \parallel A_2 \parallel A_3 \parallel s_{\text{VOLE}} \parallel t_{\text{VOLE}}$ , where  $A_0, A_1, A_2, A_3, s_{\text{VOLE}} \in F$  and  $t_{\text{VOLE}} \in \mathbb{F}_{2^{64}}$ 
2:  $H_\lambda \leftarrow 0 \in F$ ;  $H_{64} \leftarrow 0 \in \mathbb{F}_{2^{64}}$ 
3: for  $r = 0$  to  $n_\lambda - 1$  do
4:    $H_\lambda \leftarrow s_{\text{VOLE}} H_\lambda \oplus C_r(X)$ 
5:   for  $j = 0$  to  $n_{64} - 1$  do
6:      $H_{64} \leftarrow t_{\text{VOLE}} H_{64} \oplus W_{r,j}(X)$ 
7:  $Y_0 \leftarrow A_0 H_\lambda \oplus A_1 t_{64}(H_{64})$ 
8:  $Y_1 \leftarrow A_2 H_\lambda \oplus A_3 t_{64}(H_{64})$ 
9:  $Y \leftarrow \text{Ser}_F(Y_0) \parallel \text{Ser}_F(Y_1)[0..B)$ 
10: return  $Y \oplus \text{Tail}(X)$ 
```

In Algorithm 15, Ser_F is the little-endian field-element serialization from Section 4.1.3. The

slice $\text{Ser}_F(Y_1)[0..B)$ is the first $B = 16$ serialized bits, equivalently the first two little-endian bytes. The output is a byte string, not a field element.

Algorithm 16 $\text{VOLEHash.Prove}(chall_1, u, V)$

```

1:  $\delta_{bits} \leftarrow \lambda - w_{grind}$ 
2:  $\pi_{\text{VOLE}} \leftarrow \text{VOLEColumnHash}_{chall_1}(u)$ 
3:  $T_{\text{VOLE}} \leftarrow \epsilon$ 
4: for  $j = 0$  to  $\delta_{bits} - 1$  do
5:    $H_j \leftarrow \text{VOLEColumnHash}_{chall_1}(V[*, j])$ 
6:   Append  $H_j$  to  $T_{\text{VOLE}}$ 
7: for  $j = \delta_{bits}$  to  $\lambda - 1$  do
8:   Append  $\text{Tail}(V[*, j])$  to  $T_{\text{VOLE}}$ 
9: return  $(\pi_{\text{VOLE}}, T_{\text{VOLE}})$ 

```

Algorithm 17 $\text{VOLEHash.Verify}(chall_1, \pi_{\text{VOLE}}, Q, \Delta)$

```

1:  $\delta_{bits} \leftarrow \lambda - w_{grind}$ 
2: if  $|\pi_{\text{VOLE}}| \neq \lambda + B$  then
3:   return  $\perp$ 
4:  $T_{\text{VOLE}} \leftarrow \epsilon$ 
5: for  $j = 0$  to  $\delta_{bits} - 1$  do
6:    $H \leftarrow \text{VOLEColumnHash}_{chall_1}(Q[*, j])$ 
7:   if bit  $j$  of  $\Delta$  is 1 then
8:      $H \leftarrow H \oplus \pi_{\text{VOLE}}$ 
9:   Append  $H$  to  $T_{\text{VOLE}}$ 
10: for  $j = \delta_{bits}$  to  $\lambda - 1$  do
11:   Append  $\text{Tail}(Q[*, j])$  to  $T_{\text{VOLE}}$ 
12: return  $T_{\text{VOLE}}$ 

```

Since λ is a multiple of eight and $B = 16$, the VOLEHash proof is byte-aligned. There are no unused high bits and therefore no additional canonicity condition beyond the fixed-length parsing of π_{VOLE} and the surrounding signature fields.

6.4 Fiat–Shamir Challenges and Grinding

The Fiat–Shamir values are derived in three stages:

$$\begin{aligned}
chall_1 &= \text{H}_{\text{FS1}}(\mu, h, c, iv), \\
chall_2 &= \text{H}_{\text{FS2}}(chall_1, \pi_{\text{VOLE}}, T_{\text{VOLE}}, d), \\
\Delta &= \text{H}_{\text{FS3}}(chall_2, \chi_{\text{QS}}, \pi_{\text{QS}}, ctr).
\end{aligned}$$

The final challenge must satisfy two public predicates. First, the last w_{grind} bits must satisfy the grinding rule. Second, the decoded opening indices I_{Δ} must be representable by a BAVC opening of at most T_{open} seeds. The signer increments the 32-bit counter ctr until both predicates hold.

Algorithm 18 DecodeOpenChallenge(Δ)

```
1: if any bit  $\Delta[\lambda - w_{grind}], \dots, \Delta[\lambda - 1]$  is nonzero then
2:   return  $\perp$ 
3:  $pos \leftarrow 0$ 
4: for  $i = 0$  to  $\tau - 1$  do
5:    $i_\Delta[i] \leftarrow \sum_{d=0}^{k_i-1} \Delta[pos + d]2^d$ 
6:    $pos \leftarrow pos + k_i$ 
7: return  $I_\Delta$ 
```

6.5 QuickSilver Interface

The QuickSilver prover and verifier algorithms specific to the GALA relation are Algorithms 6 and 7. They invoke Algorithm 4, which returns the five degree-3 constraints for the GALA relation, and Algorithm 5, which computes the QuickSilver universal hash. The signing and verification algorithms in Section 7 treat these as black-box subroutines with the displayed inputs and outputs.

7 Signature Scheme and Serialization

7.1 Key Generation

Key generation samples a secret key k and a public input x until the validity predicate $\text{Good}(k, x)$ holds. It then evaluates the GALA OWF and outputs $pk = x \parallel y$ and $sk = k$.

Algorithm 19 KeyGen(1^λ)

```
1: repeat
2:    $k \xleftarrow{\$} F$ ; set the first two serialized bits of  $k$  to 1
3:    $x \xleftarrow{\$} F$ 
4:    $good \leftarrow \text{Good}(k, x)$ 
5: until  $good = 1$ 
6:  $y \leftarrow \text{GalaEval}(k, x)$ 
7:  $pk \leftarrow x \parallel y$ 
8:  $sk \leftarrow k$ 
9: return  $(pk, sk)$ 
```

Table 7: Normative key serialization.

Object	Field	Length	Encoding
pk	x	λ bits	field element, little-endian
	y	λ bits	field element, little-endian
sk	k	λ bits	field element, little-endian

7.2 Signing

The signing input is the public key, the secret key, and the message. The signer does not recompute or test $\text{Good}(k, x)$: keys output by Algorithm 19 already satisfy the predicate. The public key is used to obtain x and y , and the secret key is only k .

GALAS signing is deterministic. There is no signing-randomness input and no randomized signing variant in this specification. For a fixed (pk, sk, M) , the message digest, VOLE root

seed, IV preimage, BAVC/VOLE correlations, Fiat–Shamir challenges, and the first acceptable grinding counter are all derived deterministically by Algorithm 21; hence the serialized signature is uniquely determined by (pk, sk, M) .

Algorithm 20 BuildWitness(pk, sk)

```

1: Parse  $pk = x \parallel y$  and  $sk = k$ 
2:  $a_0 \leftarrow M_0(x \oplus k \oplus c_0)$ 
3:  $a_1 \leftarrow M_1(k \oplus c_1)$ 
4:  $a_2 \leftarrow M_2(k \oplus c_2)$ 
5:  $\sigma_0 \leftarrow \text{InvNormWitness}(a_0)$ 
6:  $\sigma_1 \leftarrow \text{InvNormWitness}(a_1)$ 
7:  $\sigma_2 \leftarrow \text{InvNormWitness}(a_2)$ 
8: return  $w = (k, \sigma_0, \sigma_1, \sigma_2)$ 

```

Algorithm 21 Sign(pk, sk, M)

```

1: Parse  $pk = x \parallel y$  and  $sk = k$ 
2:  $w \leftarrow \text{BuildWitness}(pk, sk)$ 
3:  $\mu \leftarrow H_{\text{msg}}(pk, M)$ 
4:  $(K_{\text{root}}, iv_{\text{pre}}) \leftarrow H_{\text{seed}}(k, \mu)$ 
5:  $iv \leftarrow H_{\text{iv}}(iv_{\text{pre}})$ 
6:  $(h, c, u, V, st_{\text{BAVC}}) \leftarrow \text{VOLE.Commit}(K_{\text{root}}, iv)$ 
7:  $chall_1 \leftarrow H_{\text{FS1}}(\mu, h, c, iv)$ 
8:  $d \leftarrow w \oplus u[0..\ell)$ 
9:  $(\pi_{\text{VOLE}}, T_{\text{VOLE}}) \leftarrow \text{VOLEHash.Prove}(chall_1, u, V)$ 
10:  $chall_2 \leftarrow H_{\text{FS2}}(chall_1, \pi_{\text{VOLE}}, T_{\text{VOLE}}, d)$ 
11:  $(\pi_{\text{QS}}, \chi_{\text{QS}}) \leftarrow \text{QS.Prove}(chall_2, w, u, V, pk)$ 
12: if  $(\pi_{\text{QS}}, \chi_{\text{QS}}) = \perp$  then
13:   return  $\perp$ 
14: for  $ctr = 0$  to  $2^{32} - 1$  do
15:    $\Delta \leftarrow H_{\text{FS3}}(chall_2, \chi_{\text{QS}}, \pi_{\text{QS}}, ctr)$ 
16:    $I_{\Delta} \leftarrow \text{DecodeOpenChallenge}(\Delta)$ 
17:   if  $I_{\Delta} = \perp$  then
18:     continue
19:    $\pi_{\text{BAVC}} \leftarrow \text{BAVC.Open}(st_{\text{BAVC}}, I_{\Delta})$ 
20:   if  $\pi_{\text{BAVC}} \neq \perp$  then
21:     return  $c \parallel \pi_{\text{VOLE}} \parallel d \parallel \pi_{\text{QS}} \parallel \pi_{\text{BAVC}} \parallel \Delta \parallel iv_{\text{pre}} \parallel ctr$ 
22: return  $\perp$ 

```

Table 8: Signature serialization order.

Field	Length	Description
c	$(\tau - 1)\hat{\ell}$ bits	VOLE correction strings
π_{VOLE}	$\lambda + B$ bits	VOLE consistency proof
d	ℓ bits	masked extended witness
π_{QS}	2λ bits	QuickSilver proof material
π_{BAVC}	$(2\tau + T_{\text{open}})\lambda$ bits	fixed BAVC opening with zero padding
Δ	λ bits	final challenge
iv_{pre}	$n_{\text{iv}} = 160$ bits	IV preimage
ctr	32 bits	little-endian grinding counter

The BAVC field is always serialized at the fixed length shown in Table 8. If the canonical opening uses fewer than T_{open} sibling seeds, the remaining seed slots are all zero; a verifier rejects any nonzero padding as specified in Algorithm 10.

7.3 Verification

Verification reconstructs the verifier side of the same transcript. It never learns k , w , or u . Instead it reconstructs $Q = V \oplus \Delta u$ from the BAVC opening and checks that the VOLEHash, QuickSilver, and final Fiat–Shamir equations are all consistent.

Algorithm 22 $\text{Verify}(pk, M, \sigma)$

```

1: Parse  $pk = x \parallel y$ 
2: Parse  $\sigma = c \parallel \pi_{\text{VOLE}} \parallel d \parallel \pi_{\text{QS}} \parallel \pi_{\text{BAVC}} \parallel \Delta \parallel iv_{\text{pre}} \parallel ctr$ 
3: if parsing fails or any field has the wrong length then
4:   return 0
5:  $I_{\Delta} \leftarrow \text{DecodeOpenChallenge}(\Delta)$ 
6: if  $I_{\Delta} = \perp$  then
7:   return 0
8:  $iv \leftarrow H_{iv}(iv_{\text{pre}})$ 
9:  $\mu \leftarrow H_{\text{msg}}(pk, M)$ 
10:  $(h, Q) \leftarrow \text{VOLE.Reconstruct}(iv, I_{\Delta}, \pi_{\text{BAVC}}, c)$ 
11: if  $h = \perp$  then
12:   return 0
13:  $chall_1 \leftarrow H_{\text{FS1}}(\mu, h, c, iv)$ 
14:  $T_{\text{VOLE}} \leftarrow \text{VOLEHash.Verify}(chall_1, \pi_{\text{VOLE}}, Q, \Delta)$ 
15:  $chall_2 \leftarrow H_{\text{FS2}}(chall_1, \pi_{\text{VOLE}}, T_{\text{VOLE}}, d)$ 
16:  $\chi_{\text{QS}} \leftarrow \text{QS.Verify}(chall_2, pk, d, Q, \Delta, \pi_{\text{QS}})$ 
17: if  $\chi_{\text{QS}} = \perp$  then
18:   return 0
19:  $\Delta' \leftarrow H_{\text{FS3}}(chall_2, \chi_{\text{QS}}, \pi_{\text{QS}}, ctr)$ 
20: if  $\Delta' = \Delta$  then
21:   return 1
22: else
23:   return 0

```

8 Parameters, Tests, and Performance

8.1 Parameters

8.1.1 Parameter Roles

Each GALAS instance is determined by the field size λ , the QuickSilver degree $d_{\text{QS}} = 3$, the VOLE/BAVC repetition count τ , the grinding width w_{grind} , and the BAVC opening cap T_{open} . The algebraic relation fixes

$$m = \lambda/2, \quad \ell = 5\lambda/2, \quad B = 16 \text{ bits}, \quad \hat{\ell} = \ell + 3\lambda + B.$$

The value B is the 16-bit universal-hash tail used by the VOLE consistency check. The BAVC leaf commitment size is 2λ bits.

8.1.2 Derived BAVC Values

The one-tree BAVC schedule uses the following notation. Let

$$k_{tree} = \left\lfloor \frac{\lambda - w_{grind}}{\tau} \right\rfloor + 1, \quad \tau_1 = (\lambda - w_{grind}) \bmod \tau, \quad \tau_0 = \tau - \tau_1.$$

Then τ_1 trees have depth k_{tree} and τ_0 trees have depth $k_{tree} - 1$. The total number of leaves is

$$L = \tau_1 2^{k_{tree}} + \tau_0 2^{k_{tree}-1}.$$

The final challenge Δ is accepted only if its last w_{grind} bits satisfy the grinding predicate and the decoded opening schedule can be represented with at most T_{open} opened λ -bit seeds. The serialized opening nevertheless reserves exactly T_{open} seed slots; unused slots are filled with zero bytes and nonzero padding is rejected.

8.1.3 Normative NGCC Parameter Sets

Table 9 gives the submitted parameter sets. Public-key and secret-key sizes are the normative scheme sizes: $pk = x \parallel y$ has 2λ bits and $sk = k$ has λ bits.

Table 9: Normative GALAS NGCC parameter sets.

Instance	λ	τ	w_{grind}	T_{open}	k_{tree}	L	pk/sk (B)	$ \sigma $ (B)
Galas-160S	160	14	7	132	11	27648	40/20	4812
Galas-160F	160	20	8	144	8	4096	40/20	5964
Galas-256S	256	21	7	218	12	79872	64/32	12114
Galas-256F	256	35	12	232	7	4416	64/32	15950
Galas-384S	384	32	6	336	12	118784	96/48	27784
Galas-384F	384	48	6	332	8	11520	96/48	33384
Galas-512S	512	42	8	456	13	172032	128/64	49516
Galas-512F	512	64	6	445	8	15616	128/64	59416

The signature sizes in the last column are obtained from the formula in Section 3.1.3. The S variants minimize the serialized transcript size for the submitted schedules. The F variants use different one-tree schedules intended to favor implementation throughput in optimized settings; actual timings are reported in Section 8.2.

8.1.4 Parameter Source

The relation parameters $\ell = 5\lambda/2$ and $d_{QS} = 3$ come from the GALA design: three wide inversion witnesses are compressed to $\lambda/2$ bits each, and all resulting constraints have QuickSilver degree at most 3. The BAVC parameters $(\tau, w_{grind}, T_{open})$ were selected by applying the one-tree BAVC size model from the VOLEitH literature to the four NGCC field sizes and then checking the resulting opening schedules in the reference and optimized implementations. The values in Table 9 are the normative submitted values.

8.2 Performance Evaluation

The NGCC package contains both a portable reference implementation and an x86-64 optimized implementation. The reference measurements use the portable C99 profile with -O2. The optimized measurements use the Meson release profile with -O3 and the x86-64 vector instructions listed in Appendix A. The self-evaluation scripts record the CPU model, operating system, compiler, compiler flags, latency, cycle counts, throughput, key sizes, signature sizes, and peak resident set size.

```

cd NGCC_Submit/Self_Evaluation
./collect_environment.sh
./run_reference_eval.sh
./run_optimized_eval.sh

```

The measurements in Tables 10 and 11 are single-run smoke benchmarks taken on June 30, 2026. The test platform was a 13th Gen Intel Core i9-13900H processor under WSL2 Ubuntu 24.04 on x86-64 with GCC 13.3.0. The figures are useful for checking scale and for detecting implementation regressions, but they are not statistically stable medians. For final reporting, regenerate the tables with a larger repetition count on the final evaluation machine.

The reference implementation is intended to be easy to audit. It uses portable C99 arithmetic and the NGCC `pseudoXOF/pseudohash` backend. The optimized implementation uses the same signature format and the same NGCC auxiliary interfaces, but replaces the main binary-field arithmetic with x86-64 optimized code paths where available.

Table 10: Reference implementation O2 smoke benchmark, one run per instance.

Instance	KeyGen (ms)	Sign (ms)	Verify (ms)	$ \sigma $ (B)
Galas-160S	5.39	690.12	407.58	4812
Galas-160F	5.07	537.39	298.76	5964
Galas-256S	14.28	3249.95	2107.89	12114
Galas-256F	11.68	2325.18	1147.83	15950
Galas-384S	30.36	10379.49	6066.71	27784
Galas-384F	37.52	8562.50	3627.62	33384
Galas-512S	60.10	23050.79	12224.36	49516
Galas-512F	49.87	19957.96	9887.33	59416

Table 11: Optimized implementation O3 smoke benchmark, one run per instance.

Instance	KeyGen (ms)	Sign (ms)	Verify (ms)	$ \sigma $ (B)
Galas-160S	2.02	516.92	120.72	4812
Galas-160F	0.30	33.14	31.50	5964
Galas-256S	1.70	1412.08	1012.72	12114
Galas-256F	0.32	79.14	78.97	15950
Galas-384S	2.30	4919.44	2097.58	27784
Galas-384F	1.05	344.83	343.36	33384
Galas-512S	4.56	15524.12	4415.24	49516
Galas-512F	2.36	586.20	708.83	59416

The optimized O3 smoke benchmark completes the key generation, signing, and verification round trip for all eight submitted instances in the same test harness. The reported numbers are from one run and should be treated as a regression check rather than a final statistical performance claim.

A large fraction of the running time in the submitted package is spent in the Fiat–Shamir, VOLE expansion, BAVC, and commitment hash/XOF layer. For the NGCC submission these calls are routed through the provided `pseudoXOF/pseudohash` auxiliary interfaces, because the package must use the submitted NGCC hash/XOF interface and because we do not assume a separate optimized production backend for all of the submitted security levels. Consequently, the reference and optimized NGCC timings mainly document interoperability, conformance, and the cost of the package as submitted. They should not be read as the final attainable performance of the underlying GALA relation once the symmetric expansion layer is implemented with a specialized high-throughput backend.

8.2.1 Optimized-Backend Reference Point

The GALAS design paper also reports an optimized-backend profile for the original 128-, 192-, and 256-bit field-size instances, using an optimized VOLEitH expansion layer rather than the NGCC auxiliary reference XOF/hash interfaces [LSP⁺26]. Those figures are not replacement parameter sets for Table 9. Their role is to show that the same VOLEitH-oriented GALA relation can be substantially faster when the symmetric expansion layer is implemented with an optimized backend.

Table 12: Illustrative optimized-backend results from the GALAS design paper [LSP⁺26].

Instance	Signature (B)	Sign (ms)	Verify (ms)	Notes
Galas-128S	2984	2.92	2.52	original field-size profile
Galas-128F	3780	0.40	0.29	original field-size profile
Galas-192S	6796	8.34	7.21	original field-size profile
Galas-192F	9728	1.00	0.70	original field-size profile
Galas-256S	12110	17.32	16.28	original field-size profile
Galas-256F	15946	2.77	1.67	original field-size profile

The NGCC benchmarks and the optimized-backend reference point measure different things. The NGCC numbers include the submitted auxiliary hash/XOF interface and are the relevant conformance figures for this package. The optimized-backend rows are included to document the performance of the underlying design when the expansion layer is implemented with a high-performance backend, and to explain why the submitted ref/opt wall-clock timings do not by themselves capture the intended performance advantage of GALAS.

8.3 KAT and Test Vectors

The submission package includes known-answer tests for all eight NGCC instances. The files are stored under NGCC_Submit/Test_Vectors/:

Instance	KAT file
Galas-160S	KAT_SIG_Galas-160S.txt
Galas-160F	KAT_SIG_Galas-160F.txt
Galas-256S	KAT_SIG_Galas-256S.txt
Galas-256F	KAT_SIG_Galas-256F.txt
Galas-384S	KAT_SIG_Galas-384S.txt
Galas-384F	KAT_SIG_Galas-384F.txt
Galas-512S	KAT_SIG_Galas-512S.txt
Galas-512F	KAT_SIG_Galas-512F.txt

Each KAT file follows the NGCC signature-vector format: it records a seed, a message, the public key, the secret key, and the signature. Because even the smallest submitted signatures are several kilobytes, this specification does not print a full signature inline. Compatibility is checked by regenerating the KAT files and by verifying each serialized signature against the recorded public key and message.

```
cd submission/reference
make kat_all
make verify_kat_all
```

```
cd submission/optimized
meson setup build --buildtype=release -Doptimization=3
ninja -C build
./build/galas_verify_kat Galas-256S < ../../kat/galas-256s.txt
```

In the packaged submission tree the same check can be run as follows:

```
cd NGCC_Submit/Implementations/Optimized_Implementation
cd Galas-256S
./build/galas_verify_kat Galas-256S < ../../kat/galas-256s.txt
./build/galas_verify_kat Galas-256S \
< ../../Test_Vectors/KAT_SIG_Galas-256S.txt
```

The KAT generation is deterministic for a fixed KAT seed and instance. The KAT seed is used to reproduce the key pair and message; the signing algorithm itself has no additional randomness. Thus a fixed recorded public key, secret key, and message always reproduce the same signature bytes. The signature bytes exercise the complete transcript format from Table 8, including VOLE correction strings, VOLEHash, QuickSilver proof material, BAVC opening data, final challenge, IV preimage, and grinding counter.

9 Security Analysis

This section analyzes the security boundary for the NGCC GALAS instances. It does not reprove the complete FAEST or VOLEitH framework. Instead it separates the inherited proof components from the GALAS-specific assumption: hardness of the single-pair degree-3 GALA relation defined in Sections 5.1–5.3.

9.1 Security Claim and Scope

This specification claims EUF-CMA security of the submitted parameter sets under the layered assumptions stated below. The claim matches the public-key distribution and the signature relation specified in this document: for a valid public key containing one pair (x, y) , producing a new accepting signature should be as hard as breaking the inherited VOLEitH/QuickSilver proof system or finding an accepting witness for the exact single-pair GALAS degree-3 relation. For witnesses satisfying $\text{Good}(k, x) = 1$, this is exactly the GALA OWF preimage statement.

The claim is intentionally narrower than a general-purpose symmetric-primitive claim. GALA is not claimed here to be a multi-user block cipher, a PRP secure under many known plaintext/ciphertext pairs, or a standalone primitive outside the GALAS signature relation. The security target and the concrete levels are summarized in Table 1; the detailed parameter accounting, hash output lengths, and quantum interpretation appear in Sections 9.7 and 9.10.

9.2 Security goal and proof boundary

The signature security goal is existential unforgeability under adaptive chosen-message attack (EUF-CMA). The Fiat–Shamir transform is analyzed in the random-oracle or quantum random-oracle style; the QROM aspect follows the standard Fiat–Shamir analysis of Don et al. [DFMS19]. The transcript structure follows the VOLEitH proof blueprint [BBdSG⁺23]. Its concrete signature layout and optimized analysis are closest to FAEST and FAESTer [BBB⁺25a, BBB⁺25b]. The concrete implementation obtains its hash and XOF calls from the NGCC auxiliary library. Section 4.2.2 states the normative domain-separation and output-length requirements; for the purpose of the proof boundary these calls are modeled as domain-separated random-oracle and pseudorandom-expansion interfaces.

Key generation samples public keys from the valid-key distribution

$$((x, y), k) \in R_{\text{GALA}}^{\text{valid}} \iff \text{Good}(k, x) = 1 \text{ and } y = \text{GALA}_k(x).$$

The proof system checks the algebraic degree-3 relation over the extended GALA witness. For honest witnesses generated from $\text{Good}(k, x) = 1$, this relation coincides with the intended single-pair OWF statement for GALA. The exact relation verified by QuickSilver is the polynomial

relation in Section 5.3. Its inverse constraints are written in the form $b_i a_i = 1$ and $z(y \oplus L_3(k)) = 1$, so an accepting witness automatically satisfies the relevant nonzero conditions. The witness used by the proof system is the extended vector

$$w = (k, \sigma_0, \sigma_1, \sigma_2) \in \{0, 1\}^{5\lambda/2},$$

where the σ_i are the subfield witnesses used to certify the three wide-round inversions.

9.3 Assumptions

The assumptions are layered as follows.

Hash/XOF model.

The domain-separated hash and XOF calls used for Fiat–Shamir challenges, tree expansion, commitments, and transcript hashing are treated as random oracles or pseudorandom generators in the same abstraction as the FAEST proof. This includes the counter-separated extension of `pseudohash` outputs and the labeled `pseudoXOF` streams specified in Section 4.2.2.

VOLEitH machinery.

The tree expansion and all-but-one vector commitment follow the GGM/one-tree style of commitment machinery [GGM84, BBM⁺24]. The VOLE correlations and consistency checks rely on the VOLEitH abstraction [BBdSG⁺23]; their concrete efficient instantiation follows the vector-OLE and silent-VOLE literature [BCGI18, Roy22]. The QuickSilver batching check is sound for authenticated polynomials of degree at most 3 [YSWW21].

GALA relation hardness.

For a random valid public key (x, y) generated by $k \xleftarrow{\$} F$, $x \xleftarrow{\$} F$, rejection sampling on $\text{Good}(k, x)$, and $y = \text{GALA}_k(x)$, it is infeasible to recover any $(k', \sigma'_0, \sigma'_1, \sigma'_2)$ satisfying the exact degree-3 GALAS constraints for (x, y) . Such a witness directly gives a conventional GALA OWF preimage: the constraints force the fixed key bits, the three wide inverses, and the final inverse tied to the public output.

Only the last assumption is GALAS-specific. The first two are the same building-block assumptions used when the FAEST relation is replaced by another low-degree relation.

9.4 Inherited reduction shape

The security proof follows the FAEST and FAESTer blueprint [BBB⁺25a, BBB⁺25b]. In GALAS, the AES relation is replaced by the exact GALA degree-3 relation specified in the GALAS design paper [LSP⁺26]. The useful way to read the reduction is not as a new proof of Fiat–Shamir, but as a compatibility statement: GALAS keeps the same transcript structure and changes only the proved relation.

Theorem 1 (Reduction shape for GALAS). *Assume the hash/XOF and VOLEitH assumptions above, and assume hardness of the exact GALAS degree-3 relation over valid public keys. For every efficient EUF-CMA forger $\mathcal{A}$ making bounded signing and oracle queries,*

$$\text{Adv}_{\text{Galas}}^{\text{EUF-CMA}}(\mathcal{A}) \leq \text{Adv}_{\text{Galas}}^{\text{EUF-KO}}(\mathcal{A}) + \text{Err}_{\text{FS/VOLE}}(\lambda, Q.),$$

where $\text{Err}_{\text{FS/VOLE}}$ is the same type of Fiat–Shamir, commitment, and opening failure term appearing in the FAEST analysis. Moreover, for efficient reductions $\mathcal{B}_1, \mathcal{B}_2$,

$$\text{Adv}_{\text{Galas}}^{\text{EUF-KO}}(\mathcal{A}) \leq \text{Adv}_{\Pi_{\text{Galas}}}^{\text{Snd}}(\mathcal{B}_2) + \text{Adv}_{\text{Gala}}^{\text{Rel}}(\mathcal{B}_1) + \text{negl}(\lambda),$$

The first step is the standard CMA-to-known-output transition: the reduction programs and answers signing queries using the same transcript distribution as the real signer, except for the usual Fiat–Shamir and commitment error terms. The second step is the false-instance switch used in VOLEitH signatures. A successful forgery on a false statement either breaks soundness of the underlying VOLEitH/QuickSilver proof system or yields an accepting witness for the exact GALAS relation. If the extracted witness also satisfies $\text{Good}(k, x) = 1$, it is a key k for the single-pair relation $y = \text{GALA}_k(x)$. With the inverse constraints in Section 5.3.4 and Section 5.3.5, accepting witnesses already force the corresponding nonzero inversion inputs; the explicit **Good** predicate remains the public-key sampling rule and a convenient way to state the honest-key distribution. The predicate $\text{Good}(k, x)$ defines the valid public-key distribution. Its rejection probability is bounded by a constant plus a negligible zero-input term, and this changes only concrete proof constants and the expected number of key-generation trials. As in the FAEST restricted-key-space treatment, this constant distributional restriction is not reported as a reduction of the nominal security level.

9.5 Role of $\text{Good}(k, x)$

The predicate $\text{Good}(k, x)$ is part of the public-key distribution and the relation-hardness boundary. Its bit-fixing clause is checked by the QuickSilver relation, while its nonzero clauses are enforced by key-generation rejection sampling and honest witness generation. It has two roles.

First, it removes the zero inputs of the four inversion operations used by GALA. Key generation rejects pairs for which one of a_0, a_1, a_2, z is zero. For the generated keys this rejection probability is negligible in λ , but making it explicit avoids any special-case inverse convention in the OWF evaluation and in honest witness generation. The submitted QuickSilver relation does not add separate high-degree nonzero tests for a_0, a_1, a_2 . Instead, the degree-3 constraints $\bar{\sigma}_i a_i^{2^m} a_i + 1 = 0$ themselves force $a_i \neq 0$ while certifying the inverse reconstruction. The final constraint $z(y \oplus L_3(k)) + 1 = 0$ similarly forces $z \neq 0$ for any accepting proof transcript. Thus the implementation keeps the public-key validity predicate and the low-degree proof relation aligned without adding signature fields.

Second, the submitted instances set the first two serialized bits of the key by requiring $k[0] = k[1] = 1$. The constraint system enforces this with $k[0] \cdot k[1] + 1 = 0$ in characteristic two. This is the same type of restricted key-space condition used in VOLEitH false-instance arguments. It fixes the valid instance distribution and affects only constant factors in sampling and reduction tightness; it is not treated as lowering the nominal λ -bit classical or $\lambda/2$ -bit Grover baseline.

9.6 Correctness and failure probability

GALAS has no decryption-style correctness failure. For honestly generated keys and honestly generated signatures, verification recomputes the same Fiat–Shamir transcript and accepts. Key generation uses rejection sampling to obtain $\text{Good}(k, x) = 1$. The fixed-bit condition means that one fourth of raw key samples pass that part of the predicate, and the probability that one of the four inversion inputs is zero is $O(2^{-\lambda})$ per trial. Thus the expected number of trials is close to four and the probability of exhausting any reasonable implementation retry bound is negligible for the submitted λ values.

The signing loop searches for a final challenge satisfying the grinding predicate and a valid BAVC opening schedule. The serialized counter is a 32-bit little-endian integer, and Algorithm 21 therefore tries the values $0, \dots, 2^{32} - 1$. If all candidate challenges miss the accepted set, the signer returns $\perp$. For the submitted grinding parameters this failure probability is negligible compared with the claimed security levels. Such an event is a signing failure, not an accepting invalid signature.

9.7 Concrete accounting of protocol parameters

The parameter table separates three effects that are sometimes easy to conflate. First, the QuickSilver check is over $F = \mathbb{F}_{2^\lambda}$ and the checked polynomial has degree at most $d_{\text{QS}} = 3$. For a fixed false polynomial before Fiat–Shamir programming losses, the usual Schwartz–Zippel term is at most $3/2^\lambda$. Second, grinding fixes only the last w_{grind} bits of the final challenge Δ so that the BAVC opening indices fit the one-tree schedule. It increases the signer’s expected number of final-hash trials and changes the set of admissible BAVC openings, but it does not change the field size of the QuickSilver check. Third, the one-tree cap T_{open} is enforced by rejection: a transcript whose opening schedule needs more than T_{open} seeds is not a valid signature.

The hash outputs used to derive the Fiat–Shamir challenges are produced by the counter-separated **pseudohash** wrapper of Section 4.2.3. The serialized BAVC root and leaf commitments remain 2λ bits even for $\lambda = 160$, where the underlying single **pseudohash** call returns a 512-bit digest. The larger Fiat–Shamir requests, such as chall_1 and chall_2 , are expanded with the same counter-separated wrapper. Thus the transcript lengths in the protocol formula are protocol lengths, not raw digest sizes.

Table 13: Baseline field, hash, and generic-search exponents for the submitted parameter sets. The fixed-bit part of **Good** is a constant distributional restriction and is not displayed as a reduced security exponent.

λ	hash digest	QS field	classical key search	Grover key search
160	512 bits	2^{160}	2^{160}	2^{80}
256	512 bits	2^{256}	2^{256}	2^{128}
384	768 bits	2^{384}	2^{384}	2^{192}
512	1024 bits	2^{512}	2^{512}	2^{256}

The fixed-bit condition in **Good** is therefore used in the same way as a restricted valid-key distribution: it may multiply the number of raw key-generation trials and the tightness constants in a reduction by an absolute constant, but it is not a separate attack path and is not subtracted from the claimed security targets.

9.8 Soundness of the GALAS relation

The QuickSilver layer checks a random linear combination of the GALAS constraints at the Fiat–Shamir challenge point $\Delta \in F$. The constraints are:

- the bit-fixing constraint on $k[0]$ and $k[1]$;
- three subfield-compressed inversion constraints $\bar{\sigma}_i a_i^{2^m} a_i + 1 = 0$;
- one final-output inversion constraint $z \cdot (y \oplus L_3(k)) + 1 = 0$.

All public maps in GALA are $\mathbb{F}_2$ -linear, and Frobenius maps such as $a \mapsto a^2$ and $a \mapsto a^{2^m}$ are also $\mathbb{F}_2$ -linear. Therefore the three subfield consistency checks are degree-3 authenticated expressions, not higher-degree S-box chains. The final-output check is also degree at most 3: z is represented as a degree-2 expression after the three subfield inversion checks, and $y \oplus L_3(k)$ is linear in the secret key. The global QuickSilver degree bound for the relation is thus $d_{\text{QS}} = 3$, which matches the payload formula used in the parameter table. The subfield-compressed inversion constraints are written in the form $b_i a_i = 1$, with $b_i = \bar{\sigma}_i a_i^{2^m}$. This form directly binds b_i to a_i^{-1} and excludes the degenerate case $a_i = 0$ without adding a separate nonzero test or increasing the degree.

9.9 GALA one-wayness analysis

The GALA OWF is used with one public input/output pair (x, y) . Attacks that require many pairs for the same key, such as classical differential, linear, higher-order differential, or cube attacks, do not directly match this single-pair signature setting. The relevant dedicated attacks are algebraic key-recovery attacks from one pair, especially linearized-polynomial decomposition and Groebner-basis solving.

9.9.1 Linearized-polynomial decomposition

For even λ , inversion over $\mathbb{F}_{2^\lambda}$ can be linearized after guessing a norm-like value. This type of algebraic leverage appears in attacks on arithmetic-oriented primitives such as Rain and AIM [ZWY⁺23]. Related resultant and polynomial-decomposition methods give further tools for studying such systems [YZY⁺24, YZY25]. The GALA analysis models whether the public linearized maps can be decomposed so that key recovery reduces to solving a low-degree univariate polynomial. Figure 2 shows the decomposition model used in the GALAS paper.

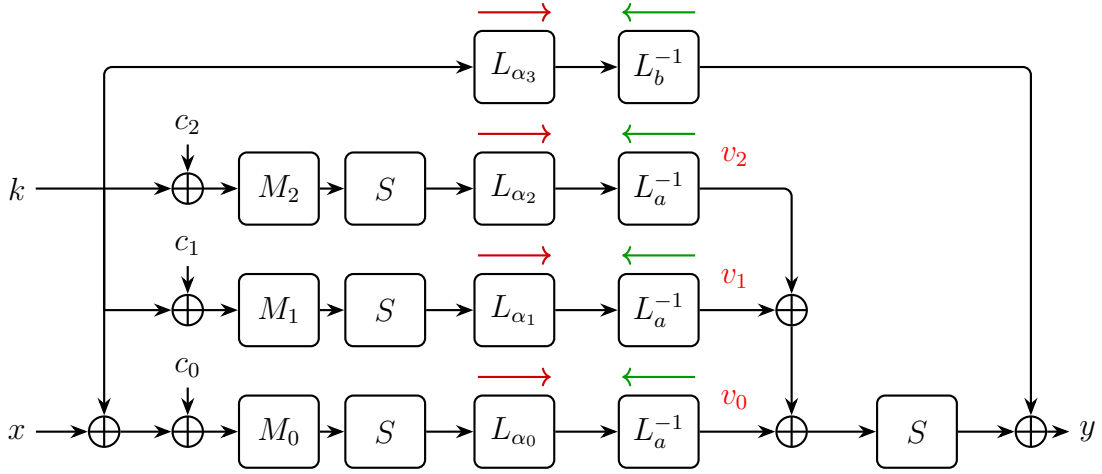


Figure 2: Linearized-polynomial decomposition model for GALA from the GALAS OWF analysis [LSP⁺26]. The red variables and the L_{α_i}, L_a, L_b maps are attack-model artifacts; they are not serialized signature values.

For dense input maps M_i , the best allocation in this model takes

$$t^* = \left\lceil \frac{2\lambda - 2}{3} \right\rceil, \quad D_M = (t^* + 2)2^{\lambda-1}.$$

The resulting attack is estimated as

$$(2^{\lambda/2} - 1) D_M \log D_M (\log D_M + \lambda \log \log D_M)$$

field operations. The important design point is that dense input maps raise the univariate degree to roughly $\lambda 2^\lambda$, while semilinear or near-semilinear relative maps would be the exceptional weak instances.

9.9.2 Weak-instance rank neighborhoods

The preceding LPD estimate assumes that the public input maps behave as dense linear maps. This assumption should be separated from an audit of fixed constants. Let

$$\mathcal{S}_1 = \{x \mapsto \gamma x^{2^r} : \gamma \in F^*, 0 \leq r < \lambda\}$$

be the semilinear family. If a relative map $R_{ij} = M_i M_j^{-1}$ lies in $\mathcal{S}_1$, then the two corresponding inverse branches have a simple Frobenius-linear relation. In that exact case, the pair can be transformed into an additional inverse component and therefore adds about 5λ quadratic Boolean equations to the direct algebraic system. This is the deterministic collapse that the map generation must avoid.

A slightly broader weak-instance condition is a low-rank neighborhood of $\mathcal{S}_1$. For an integer d , define

$$\mathcal{S}_1(d) = \{S + E : S \in \mathcal{S}_1, \text{rank}_{\mathbb{F}_2}(E) \leq d\}.$$

If several relative maps are close to semilinear and the corresponding error values depend on only a small number of key bits, then an attacker can guess that joint error and recover the same extra inverse-component equations. To make this precise, include the output key layer as a fourth node

$$N_0 = M_0, \quad N_1 = M_1, \quad N_2 = M_2, \quad N_3 = L_3,$$

and write $R_{pq} = N_p N_q^{-1}$ for $0 \leq p < q \leq 3$. For an edge set $\mathcal{E}$, suppose

$$R_{pq} = S_{pq} + E_{pq}, \quad S_{pq} \in \mathcal{S}_1 \quad ((p, q) \in \mathcal{E}).$$

The relevant guessing dimension is not the sum of the individual ranks but the joint rank

$$d_{\text{joint}}(\mathcal{E}) = \text{rank}_{\mathbb{F}_2} (k \mapsto (E_{pq} N_q(k))_{(p,q) \in \mathcal{E}}).$$

After guessing these d_{joint} bits, $M = |\mathcal{E}|$ usable edges lead heuristically to $(24 + 5M)\lambda$ quadratic equations in 4λ Boolean unknowns. Thus the condition is attack-relevant only if $C_M(\lambda) + d_{\text{joint}} < \lambda$, where $C_M(\lambda)$ is the semi-regular F4/F5 estimate for the augmented system.

For genuinely random dense invertible maps, these events are overwhelmingly unlikely. A random relative map is modeled as uniform in $\text{GL}_\lambda(2)$. Since

$$|\mathcal{S}_1| = \lambda(2^\lambda - 1), \quad |\text{GL}_\lambda(2)| = \prod_{i=0}^{\lambda-1} (2^\lambda - 2^i),$$

a union bound over the six relative maps among M_0, M_1, M_2, L_3 gives

$$\Pr[\exists \text{ exact semilinear relative map}] \leq 6 \cdot \frac{\lambda(2^\lambda - 1)}{|\text{GL}_\lambda(2)|}.$$

The rank-neighborhood event is larger, but the useful attack needs a small joint rank over multiple edges. For fixed M and D , the expected number of M -edge semilinear approximations with joint rank at most D is bounded by

$$\mathbb{E}_{M,D} \leq \binom{6}{M} (\lambda(2^\lambda - 1))^M 2^{-M\lambda^2} \sum_{r=0}^D \mathcal{N}(M\lambda, \lambda, r),$$

where $\mathcal{N}(a, b, r)$ is the number of $a \times b$ binary matrices of rank r ,

$$\mathcal{N}(a, b, r) = \prod_{i=0}^{r-1} \frac{(2^a - 2^i)(2^b - 2^i)}{2^r - 2^i}.$$

Table 14 gives representative magnitudes. The last column uses the six-edge condition and the largest integer D that would make the augmented-system estimate fall below the λ -bit classical key-search baseline. The numbers are heuristic audit estimates, not a formal reduction.

Consequently, semilinear and low-joint-rank neighborhoods are best viewed as weak-instance conditions for parameter generation and constant auditing. They do not appear to be generic attacks on randomly generated dense GALA linear layers. The submitted constants should nevertheless be checked against exact semilinear relations and small joint-rank edge sets, especially because the maps are fixed public data once the parameter set is specified.

Table 14: Random-dense-map estimates for semilinear weak-instance events. The exact column is the $\log_2$ union bound over the six relative maps among M_0, M_1, M_2, L_3 . The last column is $\log_2 \mathbb{E}_{6,D}$ for the six-edge rank-neighborhood condition.

λ	exact semilinear	six-edge D	$\log_2 \mathbb{E}_{6,D}$
160	-25428.3	5	-147019.3
256	-65267.6	36	-328414.2
384	-147059.0	69	-701667.7
512	-261618.6	116	-1167448.2

9.9.3 Groebner-basis solving

The direct algebraic model introduces inverse-output variables s_0, s_1, s_2 and writes

$$\begin{cases} s_0 M_0(x \oplus k \oplus c_0) = 1, \\ s_1 M_1(k \oplus c_1) = 1, \\ s_2 M_2(k \oplus c_2) = 1, \\ (L_0(s_0) \oplus L_1(s_1) \oplus L_2(s_2))(L_3(k) \oplus y) = 1. \end{cases}$$

Using the standard Boolean representation of inversion in $\mathbb{F}_{2^\lambda}$, this gives 20λ quadratic Boolean equations in 4λ Boolean unknowns; adding Boolean field equations $X^2 = X$ for each bit variable gives 24λ quadratic equations. The use of overdefined quadratic systems in block-cipher cryptanalysis goes back to the algebraic-attack literature, and the inversion component follows standard S-box algebraic modeling [CP02, CL04].

For a semi-regular quadratic system, the degree of regularity is estimated by the first non-positive coefficient of the Hilbert series [BFS05, Frö85]

$$S(z) = \frac{(1 - z^2)^{24\lambda}}{(1 - z)^{4\lambda}}.$$

Solving with F4/F5-style linear algebra is then estimated as

$$\binom{4\lambda + D_{\text{reg}}}{D_{\text{reg}}}^\omega$$

bit operations, with the conservative reporting choice $\omega = 2$ [Fau99, Fau02]. Table 15 gives the dedicated-attack estimates used as a sanity check for the NGCC parameters.

Table 15: Dedicated algebraic-attack estimates for the NGCC GALA field sizes. Values are $\log_2$ bit operations. The LPD estimate follows the dense-input formula above and converts one field operation to $\lambda \log \lambda$ bit operations, as in the GALAS analysis.

λ	t^*	LPD	D_{reg}	Groebner
160	106	273.7	21	261.6
256	170	420.6	30	385.9
384	256	615.2	43	561.4
512	341	808.9	55	726.5

These estimates are not used as a formal reduction. They support the relation-hardness assumption by showing that the known dedicated algebraic attacks do not beat exhaustive key search for the submitted field sizes. The parameter labels in Table 9 use the conventional nominal λ and $\lambda/2$ baselines; the fixed-bit condition in **Good** is part of the valid-instance distribution and does not change these labels.

9.9.4 Comparison with related arithmetic-oriented primitives

Rain and AIM helped establish that symmetric relations for MPCitH/VOLeith-style signatures should be evaluated under proof-system costs, not only as conventional ciphers [DKR⁺22, KHS⁺23]. At the same time, later attacks showed that arithmetic-oriented designs can have unexpected algebraic weaknesses. Groebner-style and overdefined-system attacks are represented by the Rain/AIM and quadratic-system analyses [ZWY⁺23, LMM24]; polynomial-decomposition attacks give another relevant line of analysis [YZY25]. The GALA response is to keep the inversion operation, which is excellent for subfield compression, but to separate the three wide-round inputs by dense public linear maps and to add the output key layer $L_3(k)$. This is close in spirit to strengthening patches proposed for AIM-like designs, but it is tuned for the GALAS single-pair OWF relation [SCL⁺25, LSP⁺26].

9.10 Quantum security interpretation

The quantum analysis follows the treatment of the underlying GALAS design: the relevant generic attack is Grover search over the witness/key space [Gro96]. This is a Q1-model interpretation. The adversary may use a quantum computer to speed up the classical search or algebraic computations, but the public-key distribution gives only the single GALA input/output pair (x, y) and does not provide coherent superposition access to an encryption or oracle interface under the same secret key. Multi-query period-finding attacks therefore do not match the submitted signature relation; in particular, the public maps M_i are known linear permutations and do not introduce a hidden period.

For an n -bit one-way function f , the usual Grover estimate for a preimage attack is

$$\text{Cost}_{\text{Grover}}(f) \approx 2 \left\lfloor \frac{\pi}{4} \sqrt{2^n} \right\rfloor \cdot \text{Cost}(f),$$

where $\text{Cost}(f)$ may denote size, depth, or another chosen circuit-cost metric for one evaluation of f [JBK⁺22a]. Thus the exponent is the familiar $2^{n/2}$ oracle-call exponent, while the quantum circuit for one GALA evaluation determines the constant factor.

The GALA oracle has only two operation classes: finite-field inversions and public $\mathbb{F}_2$ -linear maps. In the Toffoli/T-depth metric used for binary-field arithmetic and symmetric-key quantum-circuit estimates, all public linear maps $M_0, M_1, M_2, L_0, \dots, L_3$ can be implemented as CNOT networks. They therefore contribute no Toffoli gates and no T-depth, although they do contribute Clifford/CNOT cost. The nonlinear part contains four inversions in $\mathbb{F}_{2^\lambda}$: three wide-round inversions that can be evaluated in parallel, followed by the final inversion after their linear combination. If one inversion has Toffoli count $C_{\text{inv}}(\lambda)$ and inversion depth $D_{\text{inv}}(\lambda) = \lceil \log_2(\lambda - 1) \rceil$ as in the binary-field inversion estimates of Huang et al. [HZL25], then one GALA oracle contributes

$$4C_{\text{inv}}(\lambda) \text{ Toffoli gates and } 2D_{\text{inv}}(\lambda) \text{ inversion T-depth,}$$

up to the CNOT-only linear layers. The Toffoli decomposition model follows the standard Clifford+T accounting used in depth-oriented synthesis [AMMR13]. These costs are constants per Grover oracle call; they do not alter the $2^{\lambda/2}$ search exponent.

The original GALAS paper gives concrete Toffoli-count examples for the 128-, 192-, and 256-bit field-size instances, using optimized inversion circuits and comparing the resulting oracle costs with AES-style quantum-circuit estimates [JBK⁺22b]. Those rows are useful as design evidence but are not normative NGCC parameters, so this specification records the submitted baseline exponents and the field-size-dependent depth expression instead. The security interpretation is that quantum computation gives the expected Grover square-root speedup, while no known specialized quantum attack exploits the GALA inversion layers or the public input/output linear maps to obtain a smaller exponent.

Table 16: Quantum baseline for the submitted parameter sets. Nominal labels use the field size λ and the usual Grover square-root exponent. The fixed-bit part of **Good** is a constant distributional restriction and is not displayed as a separate reduced exponent. The last column is the inversion T-depth contribution of one GALA oracle under the circuit model above; CNOT-only linear layers are not counted in this metric.

λ	classical baseline	Grover baseline	$2D_{\text{inv}}(\lambda)$
160	160 bits	80 bits	16
256	256 bits	128 bits	16
384	384 bits	192 bits	18
512	512 bits	256 bits	18

10 Algorithm Characteristics

This section summarizes the main characteristics of the submitted GALAS scheme in the terminology used by the NGCC submission requirements. The discussion is descriptive; the formal algorithms, parameters, and security analysis are given in the previous sections.

10.1 Innovation

GALAS introduces a VOLEitH-oriented one-way function rather than reusing a general-purpose block cipher relation. The central idea is to combine finite-field inversion with subfield compression: each wide inversion is certified by a $\lambda/2$ -bit norm witness, while the entire relation remains inside a degree-3 QuickSilver budget. This targets the quantities that dominate VOLEitH signature size, namely the committed witness length ℓ and the padded payload length $\hat{\ell}$.

10.2 Simplicity

The algebraic relation has a compact structure. It uses three parallel wide-round inversions, public $\mathbb{F}_2$ -linear maps, and one final inverse tied to the public output. The proof relation contains five named constraints: one key-space constraint, three compressed inversion constraints, and one public-output constraint. The surrounding signature machinery follows the VOLEitH/QuickSilver framework, which keeps the protocol organization familiar: BAVC commitment, VOLE reconstruction, VOLEHash, QuickSilver, and Fiat–Shamir challenges.

10.3 Flexibility

The same GALA relation supports four NGCC field sizes $\lambda \in \{160, 256, 384, 512\}$ and two BAVC schedules per field size. The short and fast variants change only the VOLE/BAVC parameters $(\tau, w_{\text{grind}}, T_{\text{open}})$; the OWF, field encoding, witness layout, and constraint system remain unchanged for a given λ .

10.4 Compatibility

The submitted implementations use the NGCC signature API and the NGCC `pseudoXOF` and `pseudohash` auxiliary interfaces. The reference and optimized implementations serialize the same public keys and signatures and use the same test-vector format. At the proof-system level, the VOLEitH and QuickSilver layers use the same transcript organization as prior VOLEitH signature schemes, with the proved relation specialized to the GALA degree-3 relation.

10.5 Scalability

Scaling from $\lambda = 160$ to $\lambda = 512$ changes the field size, the subfield dimension, and the VOLE/BAVC schedule, but not the logical structure of the scheme. The witness length remains $\ell = 5\lambda/2$ and the QuickSilver degree remains $d_{\text{QS}} = 3$ for all instances. This gives a uniform path for adding or adjusting security levels, provided the field polynomial, subfield embedding basis, public linear maps, and BAVC parameters are regenerated and audited.

10.6 Implementation Performance

The reference implementation emphasizes portability and readability. The optimized implementation uses x86-64 field-arithmetic optimizations while keeping the same transcript format and NGCC auxiliary interfaces. The self-evaluation scripts report both latency and platform-independent metadata such as compiler flags, cycle counts, throughput, key/signature sizes, and peak memory usage, so performance claims can be regenerated on the final evaluation platform.

A Implementation Notes

A.1 Revision history

This version specifies the ICCS NGCC submission code and parameter sets.

- **v1 research version.** The original GALAS design used $\lambda \in \{128, 192, 256\}$, SHAKE-based hash/XOF calls, and the same degree-3 QuickSilver strategy built around subfield compression.
- **v2 ICCS NGCC submission.** The submitted fields are $\lambda = 160, 256, 384, 512$. Hash and XOF calls are instantiated through the NGCC auxiliary interfaces. The submission includes short and fast variants for each field size and enforces $\text{Good}(k, x)$ during key generation.

A.2 Reference implementation

The reference implementation (`submission/reference/`) is written in portable C99 and conforms to the NGCC `SIG_AlgorithmInstance` programming interface. It uses:

- Schoolbook $\text{GF}(2^\lambda)$ arithmetic (no SIMD).
- The NGCC `pseudoXOF` and `pseudohash` as the hash/XOF backend (via `xof.c`).
- The NGCC `drng` for randomness.

The reference implementation is designed for clarity and correctness, not speed. It includes 13 self-test suites covering GF arithmetic, OWF evaluation, BAVC commit/open/reconstruct, VOLE conversion, QuickSilver constraints, Fiat–Shamir oracles, and full sign/verify round trips for all 8 instances.

A.3 Optimized implementation

The optimized implementation (`submission/optimized/`) is derived from the FAEST/GALAS optimized code (`opt-faestAndgalas`) with the following modifications:

- Hash/PRG backend replaced: Keccak/SHAKE $\rightarrow$ NGCC `pseudoXOF/pseudohash`.
- The short and fast public instances for $\lambda \in \{160, 256, 384, 512\}$ are instantiated and tested.
- Field moduli, B' bases, and P permutations are generated for the NGCC field sizes. The optimized code uses the same irreducible polynomials as the reference implementation.

- AVX2/CLMUL field arithmetic is used where available. The optimized implementation is a single x86-64 code base covering all submitted NGCC instances, and the measured performance is reported in Section 8.2.
- KeyGen enforces $k[0] = k[1] = 1$ and S-box-nonzero rejection.
- The QuickSilver constraint $k_0 * k_1 + 1$ is used.

A.4 Performance comparison

Performance measurements for both the reference and optimized implementations are given in Section 8.2. Both implementations now use the same compact VOLEitH signature layout, so the byte counts in the reference and optimized benchmark tables are identical for each parameter set.

A.5 Code organization

File	Purpose
gf2n.c	GF(2^λ) arithmetic (add, mul, sqr, inv, frobenius)
owf.c	GALA OWF evaluation (M/L maps, inversions)
xof.c	XOF/hash layer over NGCC pseudoXOF/pseudohash
bavc.c	BAVC GGM tree commit/open/reconstruct
vole.c	Subfield VOLE conversion
universal_hashing.c	VOLEHash, ZKHash, LeafHash
quicksilver.c	QuickSilver authenticated-element arithmetic
qs_gadgets.c	OWF constraint gadgets (inverse, linearized map)
qs_full_constraints.c	Full degree-3 constraint system
qs_constraint.c	QS response computation and verification
challenge.c	final-challenge decoding and grinding check
random_oracle.c	Domain-separated hash calls $H_{\text{msg}}, H_{\text{seed}}, H_{\text{iv}}, H_{\text{com}}, H_{\text{FS1}}, H_{\text{FS2}}, H_{\text{FS3}}$; helper function names are implementation-local
sign_helpers.c	μ, K_{root} /IV derivation
SIG_AlgorithmInstance.c	NGCC interface (keygen/sign/verify)
kat_gen.c	KAT vector generator
verify_kat.c	KAT verifier
bench.c	Performance benchmark

A.6 Build instructions

```
# Reference implementation
cd submission/reference
make check          # 13 test suites
make kat_all        # 8/8 KAT generation + verification
make bench_all REPS=100

# Optimized implementation
cd submission/optimized
meson setup build --buildtype=release -Doptimization=3
ninja -C build
./build/galas_sign_test
./build/galas_bench all 100
```

```
# Packaged x86 self-evaluation scripts
cd NGCC_Submit/Self_Evaluation
./collect_environment.sh
./run_reference_eval.sh
./run_optimized_eval.sh

# Specification
cd galas-spec
latexmk -pdf -outdir=build main.tex
```

A.7 Implementation scope

1. The NGCC auxiliary hash/XOF functions `pseudoXOF` and `pseudohash` are used exactly as provided by the NGCC interface. The security discussion abstracts these calls as domain-separated pseudorandom-expansion and random-oracle interfaces. Other NGCC support code, such as the DRNG used for key generation and KAT reproducibility, is outside the Fiat–Shamir hash/XOF interface.

B Public Constant Tables

This appendix fixes the public constants used by the submitted NGCC GALAS instances. The constants are public parameters of the OWF, not run-time randomness. The source-code tables listed in Table 18 are machine-readable normative attachments: a conforming implementation may store the constants in any internal format, but after decoding it must obtain the serialized values specified here.

B.1 Canonical Serialization

The constants c_0, c_1, c_2 are serialized as field elements using Section 4.1.3. Thus byte i stores coefficients $8i, \dots, 8i+7$ of the polynomial-basis element, with coefficient $8i$ in the least significant bit of byte i .

A public linearized map

$$A(X) = \sum_{j=0}^{\lambda-1} a_j X^{2^j}$$

is serialized by the coefficient string

$$\text{SerMap}(A) = \text{Ser}_F(a_0) \parallel \text{Ser}_F(a_1) \parallel \dots \parallel \text{Ser}_F(a_{\lambda-1}).$$

The submitted C reference headers store this byte string as little-endian 64-bit limbs: coefficient a_j is the row `GALAS_λ_A[j * nw + r]`, where limb r stores bits $64r, \dots, 64r+63$. The optimized C++ header stores the same data as `A[j][r]`. The subfield-compression tables $\mathcal{B}'$ and P use the same byte order for field elements and the ordinary little-endian integer convention for the 16-bit pivot indices.

The offline SHAKE256 generation stream described in Section 5.1.1 first interprets each $\lambda/8$ -byte draw as a big-endian integer. The committed headers then serialize the resulting field element in the little-endian convention above. This conversion is part of the canonical encoding.

B.2 Round Constants

Table 17 lists the three round constants for the four submitted NGCC field sizes. Hexadecimal bytes are shown in canonical little-endian serialization order.

Table 17: GALA round constants c_0, c_1, c_2 for the NGCC instances.

λ	constant	serialized value
160	c_0	CC F4 5E AA 7B F9 18 A0 6D BA 8B 7E AF 15 13 25 00 8D 18 16
160	c_1	38 17 F5 2E 1F CA 0A 3B B1 06 55 26 9A 5B 8B 97 55 0B 86 43
160	c_2	EA D7 AD 1B 4C 55 67 44 BE 63 A4 86 9C 67 51 C2 2A F3 33 64
256	c_0	92 B4 83 06 66 72 43 30 FB 9F B7 6A EF 8E 0A 95 75 3E 6E 80 D3 CD 23 B8 44 AB B9 3D CA 4A 62 14
256	c_1	99 06 24 E7 38 23 2C AD DF 38 4A 8B 6A DF 68 CC B5 38 F7 01 6C 0A 53 67 F7 A2 F2 24 47 B6 85 D7
256	c_2	6E 5D 54 F8 D3 FC 08 5A 9F 5B B9 E1 20 F9 26 DC E6 6E 8B 27 C0 9D 37 C7 93 E2 73 78 EA 50 E1 01
384	c_0	08 43 15 C3 A1 DF BC 10 C0 78 90 65 59 4E EB FD 57 28 C6 33 D4 A1 29 C0 87 11 11 CE 31 8F 95 A0 89 9E 97 8E B4 5E 58 83 72 10 70 57 6B 10 10 5E
384	c_1	49 FF 15 8C FE BA 16 DF 9C 8A C4 71 89 07 46 9B 76 B5 BE D0 8C 33 83 A8 08 37 4A 0F 0E 27 41 6C 4C D1 D9 43 70 B4 1B D8 2A E9 0E 48 BB DA C0 E8
384	c_2	41 D6 DD BB C9 8A FC A8 11 CB AC 33 D8 EC 21 0F A4 E4 20 EF F8 40 54 E7 C6 C4 BA E5 31 B3 A7 CE E7 06 4B B1 CE 2B C1 A9 6B B4 00 F8 B2 36 64 55
512	c_0	AB 55 8C BE AD 50 7B DE 85 10 9E 61 3C B1 F0 44 5B EE 6B 5E D6 A6 62 11 A0 F9 09 85 5A 97 B1 FE DF 61 D9 B0 33 02 6B 28 BB 2D E6 FC 60 68 19 4D 06 C3 A5 74 FF 07 1F 49 37 D1 30 6A C3 DE 82 92
512	c_1	62 9B 96 0A 64 48 45 64 0E 29 9C 20 82 3E 56 2C 6B 66 C1 84 24 1B 6C 0E 9A 15 BC 59 16 3E 61 84 02 87 FE 39 7D 92 17 C0 78 FE 58 E5 8D FE D5 67 6F 3B 20 2A 44 11 25 11 CB DF FF 94 48 42 15 C0
512	c_2	FE C6 EF BE 03 96 66 78 31 07 16 04 47 E4 0B 29 D8 80 4F B2 26 8F 44 49 DB F5 19 81 8C 6E 14 68 DC B4 09 4E 2D B0 2B BC 60 B4 7A 35 05 9B 45 9C C1 14 C1 BA 05 88 FF 46 D1 A5 C0 7C 12 48 B9 91

B.3 Machine-Readable Files and Hashes

The full tables for $M_0, M_1, M_2, L_0, L_1, L_2, L_3$, together with $\mathcal{B}'$ and P , are contained in the machine-readable files listed in Table 18. The paths are relative to the source tree used to prepare the submission. In the submitted implementation package, the same files are copied into the corresponding reference and optimized instance directories. The SHA-256 values in the table version-lock the constants; they are the authoritative reference if a regenerated file differs. Spaces in the displayed hashes are inserted only for readability.

Table 18: SHA-256 hashes of the submitted public-constant files.

file	SHA-256
submission/reference/galas/galas_params_160.h	C89CEB11 0230BDF5 0FFB37C5 2CAD632B 579D4286 0D79EA79 5D065696 6810066F
submission/reference/galas/galas_params_256.h	3A524EFB 2368A2D0 F8B350ED 86628864 72CC1A8C E3535398 FDDCF5C5 EB3D951E
submission/reference/galas/galas_params_384.h	0A1FF0AF B59A041F D97244CA 77B08877 10FA06C4 1085B9D5 2E422212 CAEC562C
submission/reference/galas/galas_params_512.h	691D04B2 11EF2039 2CC71F7B 5B66686C 2AFF40EF E3980FC1 C33EC147 FB7B01C9
submission/optimized/galas_params_generated.hpp	AD6F91A6 A8CAD132 E5CC0E2E D5A9CFDC 54FDB6C9 3F8D9B2F 5FD6FF3B 7D27455A

The parameter-generation and emitter scripts used to prepare these files are identified in Table 19. These scripts are reproducibility aids; the constants themselves are locked by Table 18.

The source tree used for this specification has base Git commit

273a11eda5fc3addd686693dcb55ed53940b0be9.

Because the submission tree may contain uncommitted packaging changes, the file hashes above are the normative identifiers. Spaces in the displayed hashes are inserted only for readability.

Table 19: Hashes of the public-constant generation and emission files.

file	SHA-256
params/parameter_gen.py	E961FCB3 754DAAD9 C9276334 328AFF13 107C7630 BF7C0848 E4EA8A35 6E6F889A
params/generate_params.py	A2F042A4 5EF59F7C 909A298B A832E50E 29E909F2 CE3548BA 365F5FE3 9AF917A1
params/generated/galas_160.json	58BBAC83 10286D2D 9D8C296E 0CC5D505 E7B3AF59 71202EC9 F9FD8031 2F7EA95F
params/generated/galas_256.json	C00E30B3 542405F1 6CD7A720 C4064830 B6E93B0A BB23A306 E7C8388A 905BCF26
submission/optimized/gen_galas_params_cxx.py	516FDC03 6A02DDB0 E35752CC C3F5C136 CAB7838A 4BCE20EE E78FCC58 31549D27

References

- [AMMR13] Matthew Amy, Dmitri Maslov, Michele Mosca, and Martin Roetteler. A meet-in-the-middle algorithm for fast synthesis of depth-optimal quantum circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 32(6):818–830, 2013.
- [ARS⁺15] Martin R. Albrecht, Christian Rechberger, Thomas Schneider, Tyge Tiessen, and Michael Zohner. Ciphers for MPC and FHE. In *Advances in Cryptology – EURO-CRYPT 2015*, volume 9056 of *Lecture Notes in Computer Science*, pages 430–454. Springer, 2015.
- [BBB⁺25a] Carsten Baum, Ward Beullens, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooss, Christian Majenz, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. FAEST v2: Algorithm specifications. Technical report, NIST PQC Additional Digital Signature Submission, 2025.
- [BBB⁺25b] Carsten Baum, Ward Beullens, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooss, Christian Majenz, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. Shorter, tighter, FAESTer: Optimizations and improved (QROM) analysis for VOLE-in-the-head signatures. In *Advances in Cryptology – CRYPTO 2025*, volume 16005 of *Lecture Notes in Computer Science*, pages 124–156. Springer, 2025.
- [BBdSG⁺23] Carsten Baum, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooss, Emmanuela Orsini, Lawrence Roy, and Peter Scholl. Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. In *Advances in Cryptology – CRYPTO 2023*, volume 14085 of *Lecture Notes in Computer Science*, pages 581–615. Springer, 2023.
- [BBM⁺24] Carsten Baum, Ward Beullens, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. One tree to

- rule them all: Optimizing GGM trees and OWFs for post-quantum signatures. In *Advances in Cryptology – ASIACRYPT 2024*, volume 15484 of *Lecture Notes in Computer Science*, pages 463–493. Springer, 2024.
- [BCGI18] Elette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. Compressing vector OLE. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 896–912. ACM, 2018.
- [BFS05] Magali Bardet, Jean-Charles Faugere, and Bruno Salvy. Asymptotic behaviour of the index of regularity of semi-regular quadratic polynomial systems. In *MEGA 2005, Effective Methods in Algebraic Geometry*, 2005.
- [BNO⁺21] Carsten Baum, Daniele Nof, Eric Orsini, Peter Scholl, and Tjeerd van der Merwe. Banquet: Short and fast signatures from AES. In *Public-Key Cryptography – PKC 2021*, volume 12710 of *Lecture Notes in Computer Science*, pages 266–297. Springer, 2021.
- [CDG⁺20] Melissa Chase, David Derler, Steven Goldfeder, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, and Greg Zaverucha. The Picnic signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2020(3):1–35, 2020.
- [CL04] Jung Hee Cheon and Dong Hoon Lee. Resistance of S-boxes against algebraic attacks. In *Fast Software Encryption – FSE 2004*, volume 3017 of *Lecture Notes in Computer Science*, pages 83–94. Springer, 2004.
- [CP02] Nicolas T. Courtois and Josef Pieprzyk. Cryptanalysis of block ciphers with overdefined systems of equations. In *Advances in Cryptology – ASIACRYPT 2002*, volume 2501 of *Lecture Notes in Computer Science*, pages 267–287. Springer, 2002.
- [DFMS19] Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Security of the Fiat–Shamir transformation in the quantum random-oracle model. In *Advances in Cryptology – CRYPTO 2019*, volume 11693 of *Lecture Notes in Computer Science*, pages 356–383. Springer, 2019.
- [DKR⁺22] Christoph Dobraunig, Daniel Kales, Christian Rechberger, Markus Schofnegger, and Greg Zaverucha. Shorter signatures based on tailor-made minimalist symmetric-key crypto. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 843–857. ACM, 2022.
- [Fau99] Jean-Charles Faugere. A new efficient algorithm for computing Groebner bases (F4). *Journal of Pure and Applied Algebra*, 139(1–3):61–88, 1999.
- [Fau02] Jean Charles Faugere. A new efficient algorithm for computing Groebner bases without reduction to zero (F5). In *Proceedings of the 2002 International Symposium on Symbolic and Algebraic Computation*, pages 75–83, 2002.
- [Frö85] Ralf Fröberg. An inequality for hilbert series of graded algebras. *Mathematica Scandinavica*, 56(2):117–144, 1985.
- [FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology – CRYPTO 1986*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194. Springer, 1987.
- [GGM84] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. In *Proceedings of the 25th Annual Symposium on Foundations of Computer Science*, pages 464–479. IEEE Computer Society, 1984.

- [GMO16] Irene Giacomelli, Jesper Madsen, and Claudio Orlandi. ZKBoo: Faster zero-knowledge for boolean circuits. In *25th USENIX Security Symposium*, pages 1069–1083. USENIX Association, 2016.
- [Gro96] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, pages 212–219. ACM, 1996.
- [HZL25] Zhenyu Huang, Fuxin Zhang, and Dongdai Lin. Constructing quantum implementations with the minimal T-depth or minimal width and their applications. In *Advances in Cryptology – EUROCRYPT 2025*, volume 15601 of *Lecture Notes in Computer Science*, pages 155–185. Springer, 2025.
- [IKOS07] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Zero-knowledge from secure multiparty computation. In *Proceedings of the 39th Annual ACM Symposium on Theory of Computing*, pages 21–30. ACM, 2007.
- [JBK⁺22a] Kyungbae Jang, Anubhab Baksi, Hyunji Kim, Hwajeong Seo, and Anupam Chattopadhyay. Improved quantum analysis of SPECK and LowMC. In *Progress in Cryptology – INDOCRYPT 2022*, volume 13774 of *Lecture Notes in Computer Science*, pages 517–540. Springer, 2022.
- [JBK⁺22b] Kyungbae Jang, Anubhab Baksi, Hyunji Kim, Gyeongju Song, Hwajeong Seo, and Anupam Chattopadhyay. Quantum analysis of AES. IACR Cryptology ePrint Archive, Report 2022/683, 2022.
- [JYC⁺26] Lin Jiao, Hongsen Yang, Hongrui Cui, Yituo He, Yonglin Hao, Xiaojie Guo, Qunxiong Zheng, Jiang Zhang, Yu Yu, and Kang Yang. Lynx: Symmetric primitive for shorter and faster VOLE-in-the-head signatures. Technical Report 2026/1099, IACR Cryptology ePrint Archive, 2026.
- [KHS⁺23] Seongkwang Kim, Jincheol Ha, Mincheol Son, ByeongHak Lee, Dukjae Moon, Joohee Lee, Sangyub Lee, Jihoon Kwon, Jihoon Cho, Hyojin Yoon, and Jooyoung Lee. AIM: Symmetric primitive for shorter signatures with stronger security. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 401–415. ACM, 2023.
- [LMM24] Fukang Liu, Mohammad Mahzoun, and Willi Meier. Modelling ciphers with overdefined systems of quadratic equations: Application to friday, vision, RAIN and biscuit. In *Advances in Cryptology – ASIACRYPT 2024*, volume 15490 of *Lecture Notes in Computer Science*, pages 424–456. Springer, 2024.
- [LN97] Rudolf Lidl and Harald Niederreiter. *Finite Fields*. Cambridge University Press, 2 edition, 1997.
- [LSP⁺26] Guoxiao Liu, Yimeng Sun, Zhuoqing Peng, Chongxu Ren, Guowei Liu, Juntong Lin, Keting Jia, and Meiqin Wang. GALAS: Compact VOLE-in-the-head signatures from subfield compression. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security*, 2026. Accepted in Cycle 1; to appear.
- [Nat24a] National Institute of Standards and Technology. Module-lattice-based digital signature standard. FIPS 204, 2024.
- [Nat24b] National Institute of Standards and Technology. Post-quantum cryptography: Additional digital signature schemes. <https://csrc.nist.gov/projects/pqc-dig-sig>, 2024.

- [Roy22] Lawrence Roy. SoftSpokenOT: Quieter OT extension from small-field silent VOLE in the minicrypt model. In *Advances in Cryptology – CRYPTO 2022*, volume 13507 of *Lecture Notes in Computer Science*, pages 657–687. Springer, 2022.
- [SCL⁺25] Yimeng Sun, Shiyao Chen, Guowei Liu, Meiqin Wang, and Chao Niu. High exponents may not suffice to patch AIM: On attacks, weak parameters, and patches for AIM2. IACR Cryptology ePrint Archive, Report 2025/2272, 2025.
- [Sho94] Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, pages 124–134. IEEE Computer Society, 1994.
- [YSWW21] Kang Yang, Pratik Sarkar, Chenkai Weng, and Xiao Wang. QuickSilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2986–3001. ACM, 2021.
- [YZY⁺24] Hong-Sen Yang, Qun-Xiong Zheng, Jing Yang, Quan-Feng Liu, and Deng Tang. A new security evaluation method based on resultant for arithmetic-oriented algorithms. In *Advances in Cryptology – ASIACRYPT 2024*, volume 15490 of *Lecture Notes in Computer Science*, pages 457–489. Springer, 2024.
- [YZY25] Hong-Sen Yang, Qun-Xiong Zheng, and Jing Yang. Algebraic cryptanalysis of AO primitives based on polynomial decomposition: Applications to Rain and full AIM-I/III/V. In *Advances in Cryptology – ASIACRYPT 2025*, pages 383–410. Springer, 2025. See also IACR Cryptology ePrint Archive, Report 2025/981.
- [ZWY⁺23] Kaiyi Zhang, Qingju Wang, Yu Yu, Chun Guo, and Hongrui Cui. Algebraic attacks on round-reduced Rain and full AIM-III. In *Advances in Cryptology – ASIACRYPT 2023*, volume 14440 of *Lecture Notes in Computer Science*, pages 285–310. Springer, 2023.